PREMIUM STUDY GUIDE

MASTER SQL IN 30 DAYS

Beginner to Pro — A Complete Day-by-Day Course for Learners & Teachers

Beginner to Advanced | Queries · Joins · Design · Transactions · Optimization
Includes worked SQL examples with real output, practice exercises, and 2 full projects

30	2	90+	30
DAILY LESSONS	FULL PROJECTS	SQL EXAMPLES	TEACHER'S NOTES

"SQL is declarative — describe WHAT data you want, not the step-by-step HOW to get it."

Table of Contents

WEEK 1

SQL Foundations

Days 1–7: Understand databases and tables, write your first SELECT queries, filter, sort, and summarize data.

Why This Week Matters

Every advanced SQL skill — multi-table joins, window functions, database design — is built entirely out of this week's ideas combined in bigger ways. A shaky grasp of SELECT, WHERE, or how NULL behaves doesn't disappear as the course progresses; it resurfaces as confusing, silently-wrong query results weeks later. Students who feel tempted to rush through Week 1 because it "seems easy" are exactly the ones who benefit most from slowing down here.

This Week's Lessons

DAY 1	Introduction to SQL & Databases	Tables, rows, columns, setup
DAY 2	SELECT Basics	Choosing columns, aliases
DAY 3	Filtering with WHERE	Comparison & logical operators
DAY 4	Sorting & Limiting	ORDER BY, LIMIT, DISTINCT
DAY 5	NULL & Built-in Functions	IS NULL, COALESCE, string/number functions
DAY 6	Aggregate Functions	COUNT, SUM, AVG, MIN, MAX
DAY 7	GROUP BY & HAVING	Summarizing groups of rows + Week 1 project

By the End of This Week, You'll Be Able To

- Explain what a table, row, column, and primary key are in plain language
- Write a SELECT query that chooses specific columns and filters rows with WHERE
- Sort and limit query results, and remove duplicates with DISTINCT
- Correctly handle NULL values instead of being tripped up by them
- Summarize data using aggregate functions combined with GROUP BY and HAVING

> **🎓 Teacher's Note**
>
> If you're teaching this week to a group, resist the urge to move fast just because the syntax looks simple on the surface. The habits formed in these first seven days — reading a query in plain English before running it, double-checking how NULL behaves in a condition — are the same habits that prevent hours of confusion once joins and subqueries enter the picture in Week 2. Slow is smooth, and smooth is fast.

WEEK 1 — Foundations • DAY 1 OF 30

Introduction to SQL & Databases

> **🎓 Teacher's Note**
>
> Most beginners picture SQL as just another programming language, but it's fundamentally different: it's declarative, meaning you describe WHAT data you want, not the step-by-step HOW to get it. I always spend real time on this distinction early, because students coming from Python or JavaScript keep trying to write SQL like a procedural script, and it fights them the whole way until the declarative mindset clicks.

What Is a Database?

A **database** is an organized collection of data stored so it can be easily accessed, managed, and updated. A **relational database** (the kind SQL is built for) organizes data into **tables** — similar to spreadsheets, but with strict structure and powerful ways to connect tables together.

What Is SQL?

SQL (Structured Query Language) is the standard language used to communicate with relational databases — to create tables, insert data, and ask questions of that data. Unlike Python, SQL is **declarative**: you describe *what* data you want, not the step-by-step instructions for how to retrieve it.

Tables, Rows, and Columns

A table stores data in a grid, just like a spreadsheet:

Term	Meaning	Spreadsheet equivalent
Table	A collection of related data	One sheet/tab
Row (record)	One single entry in the table	One row
Column (field)	One property/attribute all rows share	One column header
Primary Key	A column that uniquely identifies each row	A unique ID number

Example Table: students

id	name	age	grade
1	Aisha	21	A
2	Ravi	22	B
3	Meera	20	A

Here, **id** is the primary key — no two rows can ever share the same id, which guarantees every row can be uniquely identified.

Popular Relational Database Systems

System	Common use case
SQLite	Lightweight, file-based, great for learning and small apps
MySQL	Widely used for websites and web applications
PostgreSQL	Advanced open-source database, popular for larger systems
Microsoft SQL Server	Common in enterprise/corporate environments

The good news: the **core SQL syntax taught in this course works nearly identically across all of them** — this guide uses standard SQL, with SQLite/MySQL-style examples.

Setting Up: Getting Started with SQLite

1. Download **DB Browser for SQLite** (a free visual tool) from sqlitebrowser.org, or use SQLite directly from the command line
2. Create a new database file (e.g. `school.db`)
3. You're ready to create tables and run SQL — no server installation required

💡 **TIP:** If you're teaching a classroom, SQLite is the easiest starting point — there's no server to install or configure, and every student can have their own local database file.

Your First Look at a Query

```sql
SELECT * FROM students;
```

RESULT

id	name	age	grade
1	Aisha	21	A
2	Ravi	22	B
3	Meera	20	A

3 rows returned

The * means "all columns." This query reads in plain English as: "Select everything from the students table."

✏ **PRACTICE:** Install DB Browser for SQLite (or any SQL tool your teacher recommends). Create a database file and explore its interface before writing any SQL.

Quick Check

What does a primary key guarantee about a column?

Answer: That every value in that column is unique — no two rows can share the same primary key value.

WEEK 1 — Foundations • DAY 2 OF 30

SELECT Basics

🎓 **Teacher's Note**

SELECT feels almost too simple to dwell on, but it's where students form their first mental model of what a query actually does — filtering down to columns and rows they care about. I always have students read a SELECT statement out loud in plain English before running it: 'give me the name and price columns from the products table.' That habit of narrating a query pays off enormously once WHERE, JOIN, and GROUP BY start stacking up.

The SELECT Statement

SELECT is the most common SQL command — it retrieves data from a table. Every SELECT query has two required parts: **which columns** you want, and **which table** to get them from.

```sql
SELECT column1, column2
FROM table_name;
```

Selecting All Columns

```sql
SELECT * FROM students;
```

RESULT

id	name	age	grade
1	Aisha	21	A
2	Ravi	22	B
3	Meera	20	A

Selecting Specific Columns

```sql
SELECT name, grade FROM students;
```

RESULT

name	grade
Aisha	A
Ravi	B
Meera	A

Only requesting the columns you actually need is good practice — it makes results easier to read and queries faster on large tables.

Column Aliases with AS

An **alias** temporarily renames a column in the result, without changing anything in the actual table.

```sql
SELECT name AS student_name, grade AS final_grade
FROM students;
```

RESULT

student_name	final_grade
Aisha	A
Ravi	B
Meera	A

Using Expressions in SELECT

You can compute values directly inside a SELECT — not just retrieve raw columns.

```sql
SELECT name, age, age + 1 AS age_next_year
FROM students;
```

RESULT

name	age	age_next_year
Aisha	21	22
Ravi	22	23
Meera	20	21

Selecting a Constant or Calculation

```sql
SELECT 5 + 3 AS total;
```

RESULT

total
8

You can even run SELECT without referencing any table at all — useful for quick calculations or testing.

⚠ **WATCH OUT:** SQL keywords like SELECT and FROM are traditionally written in UPPERCASE by convention (not a strict requirement in most databases) — this makes queries much easier to read at a glance, separating SQL syntax from your own table and column names.

SQL Statement Structure

Clause	Purpose	Required?
SELECT	Which columns to return	Yes
FROM	Which table to read from	Yes
WHERE	Which rows to include (tomorrow's lesson)	No
;	Ends the statement	Recommended

✏ **PRACTICE:** Using the students table, write a query that selects the name and age columns, renaming them student and years_old using aliases.

Quick Check

What does the * symbol mean in SELECT * FROM students?
Answer: It means "all columns" — every column in the table will be included in the result.

WEEK 1 — Foundations • DAY 3 OF 30

Filtering with WHERE

🎓 Teacher's Note

WHERE is the first place students write real logic in SQL, and the comparison-operator mistakes here are almost identical to the ones in any programming language — confusing = with a stricter equality check, or forgetting that text needs quotes and numbers don't. What trips people up more is NULL: WHERE column = NULL silently returns nothing, with no error, which is a much sneakier bug than a crash.

Filtering Rows with WHERE

WHERE filters which rows are included in the result, based on a condition. Without it, a SELECT returns every row in the table.

```sql
SELECT * FROM students
WHERE grade = 'A';
```

RESULT

id	name	age	grade
1	Aisha	21	A
3	Meera	20	A

2 rows returned

Comparison Operators

Operator	Meaning	Example
=	Equal to	age = 21
!= or <>	Not equal to	grade != 'A'
>	Greater than	age > 20
<	Less than	age < 22
>=	Greater than or equal to	age >= 21
<=	Less than or equal to	age <= 21

```sql
SELECT name, age FROM students
WHERE age >= 21;
```

RESULT

name	age
Aisha	21
Ravi	22

Combining Conditions: AND, OR, NOT

```sql
SELECT * FROM students
WHERE grade = 'A' AND age > 20;
```

RESULT

id	name	age	grade
1	Aisha	21	A

1 row returned

Operator	Row included when...
AND	BOTH conditions are true
OR	AT LEAST ONE condition is true
NOT	The condition is false (inverts it)

```sql
SELECT * FROM students
WHERE grade = 'A' OR age < 21;
```

RESULT

id	name	age	grade
1	Aisha	21	A
3	Meera	20	A

2 rows returned

The BETWEEN and IN Operators

```sql
SELECT name, age FROM students
WHERE age BETWEEN 20 AND 21;
```

RESULT

name	age
Aisha	21
Meera	20

```sql
SELECT name, grade FROM students
WHERE grade IN ('A', 'B');
```

RESULT

name	grade
Aisha	A
Ravi	B
Meera	A

IN is a clean shortcut for multiple OR conditions on the same column — `grade IN ('A','B')` instead of `grade = 'A' OR grade = 'B'`.

Pattern Matching with LIKE

```sql
SELECT name FROM students
WHERE name LIKE 'A%';
```

RESULT

name
Aisha

1 row returned

Wildcard	Meaning
%	Matches zero or more characters
_	Matches exactly one character

⚠ **WATCH OUT:** Text values in SQL must be wrapped in single quotes ('A', 'Aisha') — numbers are not. Mixing this up is one of the most common beginner syntax errors.

✏ **PRACTICE:** Write a query that selects all students whose age is between 18 and 21 AND whose name starts with the letter 'A' or 'M', using BETWEEN, LIKE, and OR/AND together.

Quick Check

What's the difference between = and LIKE?
Answer: = requires an exact match; LIKE allows pattern matching using wildcards like % and _.

WEEK 1 — Foundations • DAY 4 OF 30

Sorting & Limiting: ORDER BY, LIMIT, DISTINCT

🎓 **Teacher's Note**

Sorting and limiting feel like a minor convenience, but ORDER BY is where students first realize that a database table has NO inherent order — rows come back in whatever order the database feels like, unless you explicitly ask for a sort. That realization surprises almost everyone the first time, and it's worth pausing on rather than rushing past.

Sorting Results with ORDER BY

A table has no guaranteed row order — **ORDER BY** is how you explicitly control the order results come back in.

```sql
SELECT name, age FROM students
ORDER BY age;
```

RESULT

name	age
Meera	20
Aisha	21
Ravi	22

Sorted ascending (default)

```sql
SELECT name, age FROM students
ORDER BY age DESC;
```

RESULT

name	age
Ravi	22
Aisha	21
Meera	20

Sorted descending

Keyword	Meaning
ASC	Ascending (smallest/earliest first) — the default, doesn't need to be written
DESC	Descending (largest/latest first)

Sorting by Multiple Columns

```sql
SELECT name, grade, age FROM students
ORDER BY grade ASC, age DESC;
```

RESULT

name	grade	age
Aisha	A	21
Meera	A	20
Ravi	B	22

Sorted by grade first, then age within each grade

The second column only matters for breaking ties within the first — here, both A students are grouped together, then sorted by age within that group.

Limiting Results with LIMIT

LIMIT restricts how many rows come back — essential for previewing large tables or finding "top N" results.

```sql
SELECT name, age FROM students
ORDER BY age DESC
LIMIT 2;
```

RESULT

name	age
Ravi	22
Aisha	21

Top 2 oldest students

Removing Duplicates with DISTINCT

```sql
SELECT grade FROM students;
```

RESULT

grade
A
B
A

3 rows (with a duplicate)

```sql
SELECT DISTINCT grade FROM students;
```

RESULT

grade
A
B

2 unique values

DISTINCT removes duplicate rows from the result — useful for quickly seeing what unique values exist in a column.

Putting It Together

```sql
SELECT DISTINCT grade FROM students
ORDER BY grade
LIMIT 5;
```

RESULT

grade
A
B

Clause Order Matters

Clause order in your query	Clause order SQL actually runs in
SELECT	FROM
FROM	WHERE
WHERE	SELECT
ORDER BY	ORDER BY
LIMIT	LIMIT

⚠ **WATCH OUT:** SQL clauses must appear in a fixed order when written: SELECT, FROM, WHERE, ORDER BY, LIMIT. Writing ORDER BY before WHERE, for example, causes a syntax error — even though SQL logically evaluates WHERE before SELECT internally.

✏ **PRACTICE:** Write a query that selects distinct grades from the students table, sorted alphabetically, limited to the top 1 result.

Quick Check

What does LIMIT 3 do to a query's results?
Answer: It restricts the output to only the first 3 rows, based on whatever order the results are in.

WEEK 1 — Foundations • DAY 5 OF 30

NULL Values & Built-in Functions

Teacher's Note

NULL is, in my experience, the single most misunderstood concept in beginner SQL — it doesn't mean zero, it doesn't mean an empty string, it means 'unknown or missing.' That's why comparisons against NULL behave so strangely. I like to compare it to an empty box on a form versus a box where someone wrote '0' — very different pieces of information, even though both might look 'empty' at a glance.

What Is NULL?

NULL represents missing or unknown data — it is NOT the same as zero, an empty string, or false. It means "we don't have a value here."

id	name	email
1	Aisha	aisha@example.com
2	Ravi	NULL
3	Meera	meera@example.com

Ravi's email isn't an empty string — it's simply unknown or not provided.

Why = NULL Doesn't Work

```sql
SELECT * FROM students
WHERE email = NULL;
```

RESULT

id	name	email

0 rows returned — even though NULL rows exist!

This is the single most common NULL-related bug: **NULL is never equal to anything, not even another NULL**. You must use special operators instead.

IS NULL and IS NOT NULL

```sql
SELECT * FROM students
WHERE email IS NULL;
```

RESULT

id	name	email
2	Ravi	NULL

1 row returned

```sql
SELECT * FROM students
WHERE email IS NOT NULL;
```

RESULT

id	name	email
1	Aisha	aisha@example.com
3	Meera	meera@example.com

2 rows returned

COALESCE — Substituting a Default for NULL

COALESCE returns the first non-NULL value from a list — commonly used to display a friendly default instead of a blank.

```sql
SELECT name, COALESCE(email, 'No email on file') AS email
FROM students;
```

RESULT

name	email
Aisha	aisha@example.com
Ravi	No email on file
Meera	meera@example.com

Common String Functions

Function	Purpose	Example	Result
UPPER(x)	Converts to uppercase	UPPER('hi')	HI
LOWER(x)	Converts to lowercase	LOWER('HI')	hi
LENGTH(x)	Length of a string	LENGTH('hello')	5
TRIM(x)	Removes leading/trailing spaces	TRIM(' hi ')	hi
SUBSTR(x,start,len)	Extracts part of a string	SUBSTR('hello',1,3)	hel
x || y	Concatenates strings (SQLite/Postgres)	'Hi' || ' there'	Hi there

```sql
SELECT UPPER(name) AS name_upper, LENGTH(name) AS name_length
FROM students;
```

RESULT

name_upper	name_length
AISHA	5
RAVI	4
MEERA	5

Common Numeric Functions

Function	Purpose	Example	Result
ROUND(x, n)	Rounds to n decimal places	ROUND(19.567, 2)	19.57
ABS(x)	Absolute value	ABS(-8)	8
x % y (or MOD)	Remainder after division	10 % 3	1

💡 **TIP:** COALESCE is one of the most quietly useful functions in everyday SQL — reach for it any time NULL values would otherwise make a report look broken or confusing to a non-technical reader.

✏ **PRACTICE:** Write a query that shows each student's name and their email, using COALESCE to display 'Not provided' for any missing email — then add a WHERE clause to find only students with a missing email using IS NULL.

Quick Check

Why does WHERE email = NULL always return zero rows?
Answer: Because NULL represents 'unknown,' and SQL considers a comparison to an unknown value to also be unknown — never true. You must use IS NULL instead of = NULL.

WEEK 1 — Foundations • DAY 6 OF 30

Aggregate Functions: COUNT, SUM, AVG, MIN, MAX

🎓 **Teacher's Note**

Aggregate functions are usually the first genuinely exciting moment in a SQL course — suddenly a table with ten thousand rows can answer a single, useful business question in one line. The catch students hit almost immediately is mixing aggregated and non-aggregated columns in the same SELECT without GROUP BY, which is exactly what tomorrow's lesson exists to resolve.

What Are Aggregate Functions?

An **aggregate function** takes many rows and collapses them into a single summary value — total sales, average score, number of customers, and so on. This is where SQL starts to feel genuinely powerful.

The Five Core Aggregate Functions

Function	Purpose
COUNT()	Counts the number of rows
SUM()	Adds up all values in a column
AVG()	Calculates the average of a column
MIN()	Finds the smallest value
MAX()	Finds the largest value

Example Table: orders

id	customer	amount
1	Aisha	250
2	Ravi	180
3	Aisha	90
4	Meera	400

COUNT — How Many Rows?

```sql
SELECT COUNT(*) AS total_orders
FROM orders;
```

RESULT

total_orders
4

SUM — Total of a Column

```sql
SELECT SUM(amount) AS total_revenue
FROM orders;
```

RESULT

total_revenue
920

AVG — Average of a Column

```sql
SELECT AVG(amount) AS average_order
FROM orders;
```

RESULT

average_order
230

MIN and MAX

```sql
SELECT MIN(amount) AS smallest, MAX(amount) AS largest
FROM orders;
```

RESULT

smallest	largest
90	400

Combining Aggregates with WHERE

WHERE filters rows **before** the aggregate function runs on them.

```sql
SELECT COUNT(*) AS big_orders
FROM orders
WHERE amount > 100;
```

RESULT

big_orders
3

COUNT(*) vs COUNT(column)

Expression	Counts
COUNT(*)	Every row, regardless of NULLs
COUNT(column)	Only rows where that specific column is NOT NULL
COUNT(DISTINCT column)	Only unique, non-NULL values in that column

```sql
SELECT COUNT(DISTINCT customer) AS unique_customers
FROM orders;
```

RESULT

unique_customers
3

Aisha counted once, despite 2 orders

⚠ **WATCH OUT:** A single SELECT combining a raw column with an aggregate function — like SELECT customer, SUM(amount) FROM orders — without GROUP BY causes an error or unpredictable results in most databases. Tomorrow's lesson on GROUP BY is exactly how to combine them correctly.

✏ **PRACTICE:** Using the orders table, write queries to find: the total number of orders, the total revenue, the average order amount, and the number of distinct customers.

Quick Check

What's the difference between COUNT(*) and COUNT(column_name)?
Answer: COUNT(*) counts every row; COUNT(column_name) only counts rows where that column is not NULL.

WEEK 1 — Foundations • DAY 7 OF 30

GROUP BY & HAVING + Week 1 Mini Project

🎓 **Teacher's Note**

GROUP BY is the concept most likely to need a physical, hands-on demonstration rather than just an explanation — I often have students physically sort index cards into piles by category before writing a single line of SQL. Once they've grouped things by hand, GROUP BY stops being abstract syntax and starts being 'the thing I just did, but automated.'

The Problem GROUP BY Solves

Aggregate functions summarize an entire table into one row. But what if you want a summary **per category** — total sales per customer, average score per class? That's exactly what **GROUP BY** does.

GROUP BY in Action

```sql
SELECT customer, SUM(amount) AS total_spent
FROM orders
GROUP BY customer;
```

RESULT

customer	total_spent
Aisha	340
Ravi	180
Meera	400

One row per unique customer

SQL groups all rows sharing the same customer value into one bucket, then applies SUM() separately within each bucket.

GROUP BY with Multiple Aggregates

```sql
SELECT customer, COUNT(*) AS order_count, SUM(amount) AS total, AVG(amount) AS average
FROM orders
GROUP BY customer;
```

RESULT

customer	order_count	total	average
Aisha	2	340	170
Ravi	1	180	180
Meera	1	400	400

Filtering Groups with HAVING

WHERE filters individual rows *before* grouping. **HAVING** filters entire groups *after* aggregation — you cannot use WHERE to filter on an aggregate result like SUM().

```sql
SELECT customer, SUM(amount) AS total_spent
FROM orders
GROUP BY customer
HAVING SUM(amount) > 200;
```

RESULT

customer	total_spent
Aisha	340
Meera	400

Only customers who spent over 200

WHERE vs HAVING — Side by Side

	WHERE	HAVING
Filters	Individual rows	Grouped/aggregated results
Runs	Before grouping	After grouping
Can use aggregate functions?	No	Yes
Example	WHERE amount > 100	HAVING SUM(amount) > 200

Combining WHERE, GROUP BY, and HAVING

```sql
SELECT customer, COUNT(*) AS order_count
FROM orders
WHERE amount > 50
GROUP BY customer
HAVING COUNT(*) >= 1
ORDER BY order_count DESC;
```

RESULT

customer	order_count
Aisha	2
Ravi	1
Meera	1

The Full Clause Order

Order written	Purpose
SELECT	Choose columns/expressions
FROM	Choose the table
WHERE	Filter individual rows
GROUP BY	Group rows into buckets
HAVING	Filter grouped results
ORDER BY	Sort the final output
LIMIT	Restrict row count

💡 **TIP:** A simple rule that resolves almost all WHERE-vs-HAVING confusion: if your condition mentions a raw column, use WHERE. If it mentions an aggregate function like SUM() or COUNT(), use HAVING.

✏ **PRACTICE:** Using the orders table, write a query that groups orders by customer, shows their total spend, and only includes customers with more than 1 order — using HAVING COUNT(*) > 1.

Quick Check

Why can't you write WHERE SUM(amount) > 200 instead of using HAVING?
Answer: Because WHERE filters rows BEFORE aggregation happens, so SUM() doesn't exist yet at that stage of query execution — HAVING runs after grouping, when aggregates are available.

Week 1 Mini Project: Sales Summary Report

Combine everything from this week into one realistic reporting query.

id	product	category	price	quantity
1	Notebook	Stationery	60	3
2	Pen	Stationery	10	10
3	Laptop	Electronics	55000	1
4	Mouse	Electronics	500	2
5	Eraser	Stationery	5	20

```sql
SELECT category,
       COUNT(*) AS num_products,
       SUM(price * quantity) AS total_value
FROM products
GROUP BY category
HAVING SUM(price * quantity) > 1000
ORDER BY total_value DESC;
```

RESULT

category	num_products	total_value
Electronics	2	56000

Only Electronics clears the HAVING > 1000 threshold — Stationery totals only 380 and is filtered out

✏ **PRACTICE:** Recreate the products table shown above in your own database, then write the sales summary query yourself before checking it against the example — this is the best way to confirm the week's concepts have really landed.

WEEK 2

Relationships & Joins

Days 8–14: Understand how tables connect, and combine data across multiple tables using every type of JOIN and subquery.

Why This Week Matters

This is the week SQL stops being about one table and starts being about how real databases actually work — data split across many related tables instead of one giant spreadsheet. Primary and foreign keys teach the database how tables connect; JOINs teach you how to pull connected data back together on demand. This single idea — splitting data apart, then recombining it precisely when needed — is the foundation of virtually every real-world database system.

This Week's Lessons

DAY 8	Relationships	Primary keys, foreign keys, ER concept
DAY 9	INNER JOIN	Combining matching rows
DAY 10	LEFT & RIGHT JOIN	Keeping unmatched rows
DAY 11	Self Joins & CROSS JOIN	Joining a table to itself
DAY 12	Subqueries	Queries inside queries
DAY 13	UNION, INTERSECT, EXCEPT	Combining result sets
DAY 14	Project 1	Library Management queries

By the End of This Week, You'll Be Able To

- Explain how a foreign key connects two tables
- Write an INNER JOIN to combine matching rows from two or more tables
- Choose correctly between INNER, LEFT, and RIGHT JOIN for a given reporting need
- Write a subquery to filter or compute a value using a separate query
- Combine multiple result sets using UNION

> **🎓 Teacher's Note**
> Watch for the moment a student starts sketching two tables on paper and drawing a line between the matching columns before writing any SQL. That visual habit — seeing the relationship before writing the JOIN — is the single biggest predictor of who's about to genuinely understand joins versus who's about to memorize syntax without the underlying picture.

WEEK 2 — Joins • DAY 8 OF 30

Understanding Relationships: Primary & Foreign Keys

> **🎓 Teacher's Note**
> Understanding primary and foreign keys is really understanding WHY tables are split apart in the first place — a concept students often haven't questioned yet, since Excel/spreadsheet thinking treats one giant table as normal. I frame it as: we split data into multiple tables specifically so it doesn't have to repeat itself, and the keys are simply how we stitch it back together on demand.

Why Split Data Across Multiple Tables?

You could store everything in one giant table, but that causes serious problems: repeated data, wasted space, and a nightmare when something needs updating in fifty places at once. Instead, relational databases split data into **separate, focused tables** and connect them using **keys**.

A Motivating Example: The Problem

order_id	customer_name	customer_email	product
1	Aisha	aisha@x.com	Notebook
2	Aisha	aisha@x.com	Pen
3	Ravi	ravi@x.com	Laptop

Aisha's name and email are repeated in every order. If her email changes, you'd need to update it in every single row — error-prone and wasteful.

The Solution: Two Related Tables

`customers`

id	name	email
1	Aisha	aisha@x.com
2	Ravi	ravi@x.com

`orders`

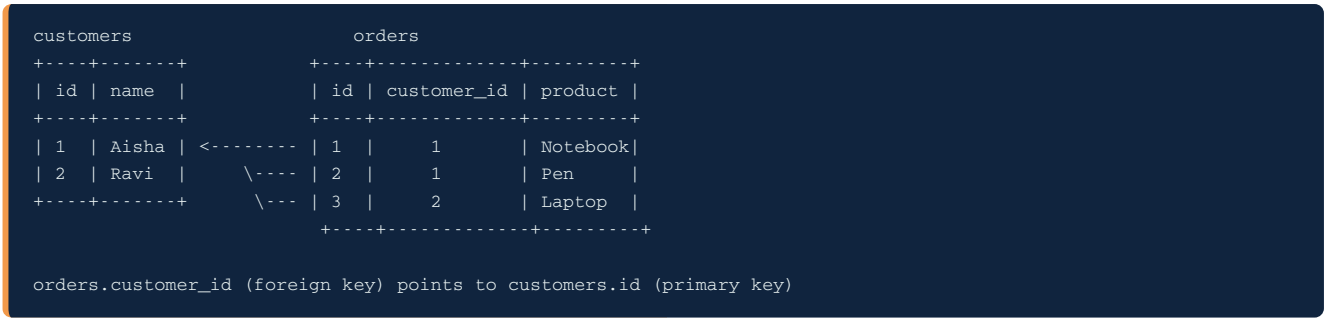
id	customer_id	product
1	1	Notebook
2	1	Pen
3	2	Laptop

Now Aisha's details exist in exactly **one** place. The **orders** table only stores her **id** (1) as a reference — called a **foreign key**.

Primary Key vs Foreign Key

Term	Meaning	In this example
Primary Key	Uniquely identifies each row in ITS OWN table	customers.id
Foreign Key	A column that refers to a primary key IN ANOTHER table	orders.customer_id

A Simple Entity-Relationship Sketch



Types of Relationships

Relationship	Example	How it's modeled
One-to-Many	One customer, many orders	Foreign key on the 'many' side (orders.customer_id)
Many-to-Many	Students and courses (each takes many, each has many students)	A separate 'junction' table linking both
One-to-One	A person and their passport	Foreign key with a UNIQUE constraint

💡 **TIP:** Before writing any query involving multiple tables, sketch the tables on paper and draw an arrow from the foreign key to the primary key it points to. This 30-second habit prevents most beginner JOIN mistakes.

✏ **PRACTICE:** Sketch out two related tables for a 'books' and 'authors' scenario: each book has exactly one author, but each author can write many books. Identify the primary key in each table and where the foreign key belongs.

Quick Check

Where does the foreign key live in a one-to-many relationship?

Answer: On the 'many' side of the relationship — e.g. each order (the many side) stores a reference to one customer (the one side).

WEEK 2 — Joins • DAY 9 OF 30

INNER JOIN

🎓 **Teacher's Note**

INNER JOIN is the single biggest conceptual leap so far in this course, comparable to functions in a programming language — I never rush it. The key reframing that helps most: a join doesn't 'merge tables' as some vague blob, it recombines specific ROWS based on a matching condition, one pair at a time. Drawing two tiny tables by hand and connecting matching rows with actual lines works better than any slide.

What Is a JOIN?

A **JOIN** combines rows from two or more tables based on a related column — typically a foreign key matching a primary key. **INNER JOIN** is the most common type: it returns only rows that have a match in **both** tables.

Our Two Tables

`customers`

id	name
1	Aisha
2	Ravi
3	Meera

`orders`

id	customer_id	product
1	1	Notebook
2	1	Pen
3	2	Laptop

Notice: Meera (id 3) has no matching orders at all.

Writing an INNER JOIN

```sql
SELECT customers.name, orders.product
FROM customers
INNER JOIN orders
  ON customers.id = orders.customer_id;
```

RESULT

name	product
Aisha	Notebook
Aisha	Pen
Ravi	Laptop

3 rows — Meera is excluded, she has no matching order

The **ON** clause defines exactly how the two tables are related — here, customers.id must equal orders.customer_id for a row to appear in the result.

Using Table Aliases for Cleaner Queries

Real queries usually alias table names to keep the SQL shorter and more readable.

```sql
SELECT c.name, o.product
FROM customers AS c
INNER JOIN orders AS o
  ON c.id = o.customer_id;
```

RESULT

name	product
Aisha	Notebook
Aisha	Pen
Ravi	Laptop

Joining Three Tables

products

id	name	price
1	Notebook	60
2	Pen	10
3	Laptop	55000

```sql
SELECT c.name AS customer, p.name AS product, p.price
FROM customers AS c
INNER JOIN orders AS o ON c.id = o.customer_id
INNER JOIN products AS p ON o.product = p.name;
```

RESULT

customer	product	price
Aisha	Notebook	60
Aisha	Pen	10
Ravi	Laptop	55000

You can chain as many JOINs as needed — each one connects one more related table into the result.

Why INNER JOIN Drops Unmatched Rows

INNER JOIN keeps a row when...
Both tables have a matching value in the joined columns
A customer with zero orders → EXCLUDED entirely
An order with an invalid customer_id → EXCLUDED entirely

⚠ **WATCH OUT:** Forgetting the ON clause — or joining on the wrong columns — is one of the most common and dangerous JOIN mistakes. Without a correct ON condition, SQL may produce a 'cross join' pairing every row with every other row, silently creating a huge, meaningless result.

✏ **PRACTICE:** Using the customers, orders, and products tables above, write an INNER JOIN query that shows each customer's name alongside the total price of everything they've ordered (hint: you'll need all three tables joined together).

Quick Check

What happens to a customer with no orders when you use INNER JOIN between customers and orders?
Answer: They're completely excluded from the result — INNER JOIN only returns rows that have a match in BOTH tables.

WEEK 2 — Joins • DAY 10 OF 30

LEFT JOIN, RIGHT JOIN & FULL OUTER JOIN

🎓 **Teacher's Note**

LEFT JOIN confuses people mainly because they haven't yet been shown a concrete situation where INNER JOIN silently drops data they actually needed — like customers with zero orders vanishing entirely from a report. I always demonstrate that missing-data problem FIRST, then introduce LEFT JOIN as the fix, rather than presenting all the join types as an abstract list to memorize.

The Problem INNER JOIN Doesn't Solve

Sometimes you specifically **want** to see rows with no match — like "which customers have never placed an order?" INNER JOIN would silently hide exactly the customers you're looking for.

LEFT JOIN — Keep Every Row from the Left Table

LEFT JOIN returns every row from the first ("left") table, and matches from the second table where they exist — filling in NULL where there's no match.

```sql
SELECT c.name, o.product
FROM customers AS c
LEFT JOIN orders AS o
  ON c.id = o.customer_id;
```

RESULT

name	product
Aisha	Notebook
Aisha	Pen
Ravi	Laptop
Meera	NULL

4 rows — Meera now appears, with NULL for product

Finding Unmatched Rows: A Very Common Report

Combine LEFT JOIN with a check for NULL to find exactly the rows with no match — a genuinely common real-world reporting need.

```sql
SELECT c.name
FROM customers AS c
LEFT JOIN orders AS o
  ON c.id = o.customer_id
WHERE o.id IS NULL;
```

RESULT

name
Meera

Customers who have never placed an order

RIGHT JOIN — The Mirror Image

RIGHT JOIN keeps every row from the second ("right") table instead, matching from the left where possible.

```sql
SELECT c.name, o.product
FROM orders AS o
RIGHT JOIN customers AS c
  ON c.id = o.customer_id;
```

RESULT

name	product
Aisha	Notebook
Aisha	Pen
Ravi	Laptop
Meera	NULL

Same result as the LEFT JOIN example above, tables swapped

In practice, most developers only ever use LEFT JOIN and simply swap which table comes first — RIGHT JOIN is rarely needed and not supported in every database (SQLite added it only in recent versions).

FULL OUTER JOIN — Keep Everything from Both Sides

FULL OUTER JOIN returns all rows from both tables, matching where possible and filling NULL where there's no match on either side. Not supported in SQLite/MySQL directly, but available in PostgreSQL and SQL Server.

```sql
SELECT c.name, o.product
FROM customers AS c
FULL OUTER JOIN orders AS o
  ON c.id = o.customer_id;
```

RESULT

name	product
Aisha	Notebook
Aisha	Pen
Ravi	Laptop
Meera	NULL

Would also show any orphaned orders with no valid customer, if they existed

Choosing the Right JOIN

Need	Use
Only rows that match in both tables	INNER JOIN
All rows from table A, matched where possible	LEFT JOIN
All rows from table B, matched where possible	RIGHT JOIN
Every row from both tables, matched where possible	FULL OUTER JOIN

💡 **TIP:** Default to LEFT JOIN whenever you're not 100% sure INNER JOIN is correct — it's much easier to notice and filter out unwanted NULLs afterward than to silently lose real data you needed to see.

✏ **PRACTICE:** Using customers and orders, write a LEFT JOIN query to list every customer along with their order count (using COUNT and GROUP BY), making sure customers with zero orders still show up with a count of 0.

Quick Check

How do you find customers with NO matching orders using a LEFT JOIN?
Answer: LEFT JOIN customers to orders, then add WHERE orders.id IS NULL — this catches exactly the customer rows where no match was found.

WEEK 2 — Joins • DAY 11 OF 30

Self Joins & CROSS JOIN

🎓 **Teacher's Note**

Self joins are genuinely one of the harder ideas to visualize, because the mental model of 'joining a table to itself' doesn't map to anything in everyday spreadsheet use. I always use an org-chart example (employee/manager) since almost every student immediately understands hierarchical relationships, even before they understand the SQL syntax describing it.

What Is a Self Join?

A **self join** joins a table to itself — useful when rows in a table relate to OTHER rows in that exact same table. The classic example: an employees table where each employee has a manager who is also an employee.

Example Table: employees

id	name	manager_id
1	Priya (CEO)	NULL
2	Ravi	1
3	Aisha	1
4	Karan	2

Karan's manager_id (2) points to Ravi's id — both rows live in the exact same table.

Writing a Self Join

```sql
SELECT e.name AS employee, m.name AS manager
FROM employees AS e
LEFT JOIN employees AS m
  ON e.manager_id = m.id;
```

RESULT

employee	manager
Priya (CEO)	NULL
Ravi	Priya (CEO)
Aisha	Priya (CEO)
Karan	Ravi

The trick: the same table is given **two different aliases** (e and m) so SQL can treat it as if it were two separate tables for the

purposes of the join.

Why LEFT JOIN, Not INNER JOIN, Here

Priya has no manager (manager_id is NULL). An INNER JOIN would silently exclude her from the results entirely — LEFT JOIN correctly keeps her with a NULL manager.

CROSS JOIN — Every Combination of Rows

A **CROSS JOIN** pairs every row from one table with every row from another — no matching condition at all. Used rarely, but useful for generating all possible combinations.

`sizes`

size
S
M
L

`colors`

color
Red
Blue

```sql
SELECT s.size, c.color
FROM sizes AS s
CROSS JOIN colors AS c;
```

RESULT

size	color
S	Red
S	Blue
M	Red
M	Blue
L	Red
L	Blue

6 rows — 3 sizes x 2 colors = every combination

When You'd Actually Use CROSS JOIN

Use case	Why CROSS JOIN fits
Generating a product variant list (size x color)	Every combination needs to exist as a row
Building a calendar of every date x every store	Combinatorial data that doesn't come from a 'relationship'

⚠ **WATCH OUT:** An accidental CROSS JOIN — usually caused by forgetting the ON clause on a regular JOIN — is a common and confusing bug. If a query's result has WAY more rows than expected, check whether a join condition was left out.

✏ **PRACTICE:** Using the employees table above, write a self join that lists every employee along with how many people report directly to them (hint: GROUP BY the manager side and COUNT).

Quick Check

Why does a self join need table aliases?
Answer: Because both 'copies' of the join refer to the same physical table — aliases let SQL (and you) distinguish which reference means what, like 'the employee' versus 'their manager.'

WEEK 2 — Joins • DAY 12 OF 30

Subqueries

🎓 Teacher's Note

Subqueries are where SQL starts to feel like it has real depth. I explicitly show students the same problem solved two ways — with a subquery and with a JOIN — so they see these aren't always two totally different tools, but often two ways to express the same idea. Knowing both means being able to choose whichever reads more clearly for a given problem.

What Is a Subquery?

A **subquery** is a query nested inside another query — used when you need the RESULT of one query as an input to another. Subqueries can appear in WHERE, SELECT, or FROM.

A Subquery in WHERE

orders

id	customer_id	amount
1	1	250
2	1	90
3	2	400

```sql
SELECT name FROM customers
WHERE id IN (
    SELECT customer_id FROM orders WHERE amount > 200
);
```

RESULT

name
Aisha
Ravi

Customers who have placed at least one order over 200

The inner query runs first, producing a list of customer_ids; the outer query then finds customers matching any of those ids.

Subquery vs JOIN — Two Ways to the Same Answer

```sql
SELECT DISTINCT c.name
FROM customers AS c
INNER JOIN orders AS o ON c.id = o.customer_id
WHERE o.amount > 200;
```

RESULT

name
Aisha
Ravi

Identical result, written as a JOIN instead

Both queries above answer the same question. Subqueries are often more readable for "filter by a condition on another table"; JOINs are usually better when you need columns from BOTH tables in the final result.

A Subquery in SELECT

```sql
SELECT name,
  (SELECT COUNT(*) FROM orders WHERE orders.customer_id = customers.id) AS order_count
FROM customers;
```

RESULT

name	order_count
Aisha	2
Ravi	1
Meera	0

This is called a **correlated subquery** — it runs once for every row of the outer query, referencing that row's value (customers.id) each time.

Subqueries with Aggregate Comparisons

```sql
SELECT name, amount FROM orders
WHERE amount > (SELECT AVG(amount) FROM orders);
```

RESULT

name	amount
Aisha	250
Meera	400

Orders above the average order amount

EXISTS — Checking for Any Match

```sql
SELECT name FROM customers AS c
WHERE EXISTS (
    SELECT 1 FROM orders AS o WHERE o.customer_id = c.id
);
```

RESULT

name
Aisha
Ravi

Customers who have placed at least one order

EXISTS is often faster than IN for large tables, since it stops checking as soon as it finds one match, rather than building a full list.

💡 **TIP:** When you're unsure whether to write a subquery or a JOIN, try both and compare — for most beginner-to-intermediate queries, correctness and readability matter far more than the small performance difference between them.

✏ **PRACTICE:** Write a subquery that finds all products priced above the average product price, then rewrite the same question using a JOIN-free approach with just a subquery in WHERE.

Quick Check

What's a correlated subquery?

Answer: A subquery that references a column from the outer query, causing it to run once per row of the outer query instead of just once overall.

WEEK 2 — Joins • DAY 13 OF 30

UNION, INTERSECT & EXCEPT

> **Teacher's Note**
> UNION trips people up mostly through the column-count and data-type matching requirement, which feels like an arbitrary restriction until you remember the result has to be one single, coherent table. I ask students to predict what error they'd get from mismatched columns BEFORE running the query — reasoning about the failure ahead of time cements the rule far better than just being told it.

Combining Results from Multiple Queries

Sometimes you need to combine the results of two separate SELECT statements into one list — for example, combining "current customers" and "past customers" into a single mailing list.

UNION — Combine and Remove Duplicates

`current_customers`

name
Aisha
Ravi

`past_customers`

name
Ravi
Meera

```sql
SELECT name FROM current_customers
UNION
SELECT name FROM past_customers;
```

RESULT

name
Aisha
Meera
Ravi

3 unique names — Ravi appears in both tables but shows only once

UNION ALL — Combine and Keep Duplicates

```sql
SELECT name FROM current_customers
UNION ALL
SELECT name FROM past_customers;
```

RESULT

name
Aisha
Ravi
Ravi
Meera

4 rows — Ravi appears twice, once from each table

	UNION	UNION ALL
Removes duplicates?	Yes	No
Performance	Slower (must check for duplicates)	Faster
Use when...	You want a clean, unique list	You want every row, including exact duplicates, or you know there are none

Rules for Using UNION

- Both SELECT statements must return the **same number of columns**
- Corresponding columns must have **compatible data types**
- Column names in the final result come from the **first** SELECT statement

```sql
SELECT name, 'Current' AS status FROM current_customers
UNION
SELECT name, 'Past' AS status FROM past_customers;
```

RESULT

name	status
Aisha	Current
Meera	Past
Ravi	Current
Ravi	Past

Adding a label column lets you tell the two sources apart

INTERSECT — Only Rows in BOTH Queries

```sql
SELECT name FROM current_customers
INTERSECT
SELECT name FROM past_customers;
```

RESULT

name
Ravi

Only Ravi appears in both tables

EXCEPT — Rows in the First Query, Not the Second

```sql
SELECT name FROM current_customers
EXCEPT
SELECT name FROM past_customers;
```

RESULT

name
Aisha

Current customers who were never past customers

Operator	Returns
UNION	Everything from both, duplicates removed
UNION ALL	Everything from both, duplicates kept
INTERSECT	Only rows appearing in BOTH results
EXCEPT	Rows in the first result that are NOT in the second

⚠ **WATCH OUT:** INTERSECT and EXCEPT are not supported in older MySQL versions (added in MySQL 8.0.31+) — check your specific database's documentation if a query using them doesn't work as expected.

✏ **PRACTICE:** Using the current_customers and past_customers tables, write all four queries: UNION, UNION ALL, INTERSECT, and EXCEPT, and compare their row counts to confirm you understand what each one returns.

Quick Check

What's the key difference between UNION and UNION ALL?

Answer: UNION removes duplicate rows from the combined result; UNION ALL keeps every row, including exact duplicates.

WEEK 2 CAPSTONE • BEGINNER PROJECT

Project 1: Library Management System

Project Overview

You're going to design and query a small **Library Management System** — a realistic multi-table database with books, members, and loans. This project uses **every skill from Week 1 and Week 2**: filtering, aggregation, and every type of JOIN.

The Schema

```
books                        members
+----+----------------+      +----+---------+
| id | title          |      | id | name    |
+----+----------------+      +----+---------+
| 1  | Python Basics  |      | 1  | Aisha   |
| 2  | SQL for All    |      | 2  | Ravi    |
| 3  | Data Stories   |      | 3  | Meera   |
+----+----------------+      +----+---------+

loans
+----+---------+-----------+------------+-------------+
| id | book_id | member_id | loan_date  | return_date |
+----+---------+-----------+------------+-------------+
| 1  |    1    |     1     | 2026-01-05 |  2026-01-19 |
| 2  |    2    |     1     | 2026-02-01 |    NULL     |
| 3  |    3    |     2     | 2026-02-10 |    NULL     |
+----+---------+-----------+------------+-------------+
NULL return_date means the book is still out on loan
```

Step 1: Create the Tables

```sql
CREATE TABLE books (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL
);

CREATE TABLE members (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);

CREATE TABLE loans (
    id INTEGER PRIMARY KEY,
    book_id INTEGER,
    member_id INTEGER,
    loan_date TEXT,
    return_date TEXT,
    FOREIGN KEY (book_id) REFERENCES books(id),
    FOREIGN KEY (member_id) REFERENCES members(id)
);
```

Step 2: Insert Sample Data

```sql
INSERT INTO books VALUES (1, 'Python Basics'), (2, 'SQL for All'), (3, 'Data Stories');
INSERT INTO members VALUES (1, 'Aisha'), (2, 'Ravi'), (3, 'Meera');
INSERT INTO loans VALUES
    (1, 1, 1, '2026-01-05', '2026-01-19'),
    (2, 2, 1, '2026-02-01', NULL),
    (3, 3, 2, '2026-02-10', NULL);
```

Query 1: Which Books Are Currently Out on Loan?

```sql
SELECT b.title, m.name AS borrowed_by, l.loan_date
FROM loans AS l
INNER JOIN books AS b ON l.book_id = b.id
INNER JOIN members AS m ON l.member_id = m.id
WHERE l.return_date IS NULL;
```

RESULT

title	borrowed_by	loan_date
SQL for All	Aisha	2026-02-01
Data Stories	Ravi	2026-02-10

Query 2: Which Members Have Never Borrowed a Book?

```sql
SELECT m.name
FROM members AS m
LEFT JOIN loans AS l ON m.id = l.member_id
WHERE l.id IS NULL;
```

RESULT

name
Meera

Query 3: How Many Times Has Each Book Been Borrowed?

```sql
SELECT b.title, COUNT(l.id) AS times_borrowed
FROM books AS b
LEFT JOIN loans AS l ON b.id = l.book_id
GROUP BY b.title
ORDER BY times_borrowed DESC;
```

RESULT

title	times_borrowed
Python Basics	1
SQL for All	1
Data Stories	1

Query 4: Which Member Has Borrowed the Most Books?

```sql
SELECT m.name, COUNT(l.id) AS total_loans
FROM members AS m
INNER JOIN loans AS l ON m.id = l.member_id
GROUP BY m.name
ORDER BY total_loans DESC
LIMIT 1;
```

RESULT

name	total_loans
Aisha	2

Extend the Project (Recommended)

- ☐ Add a query that finds books that have NEVER been borrowed, using LEFT JOIN + IS NULL
- ☐ Add a "due_date" column and write a query to find overdue loans (due_date in the past, return_date still NULL)
- ☐ Add a genre column to books and write a report grouping loan counts by genre
- ☐ Add a fines table and calculate total fines owed per member using a JOIN and SUM

💡 **TIP:** This project mirrors almost exactly what a junior data analyst gets asked in their first month on the job: 'which of X have never done Y' and 'who has done the most Z' are two of the most common real-world query patterns you'll ever write.

✏ **PRACTICE:** Build the full Library Management schema exactly as shown, run all 4 queries, then implement at least one extension from the list above on your own.

WEEK 3

Building & Modifying Databases

Days 15–21: Create and alter tables, insert/update/delete data safely, enforce data quality with constraints, and speed up queries with views and indexes.

Why This Week Matters

This week marks a shift from querying data someone else built to designing and maintaining a database yourself. CREATE TABLE means deciding the shape of your data up front. UPDATE and DELETE mean your mistakes are no longer harmless — a wrong WHERE clause can now change or destroy real data. Constraints, views, and indexes are the professional tools that keep a growing database correct, convenient, and fast.

This Week's Lessons

DAY 15	CREATE TABLE	Data types and table design
DAY 16	ALTER & DROP TABLE	Modifying table structure
DAY 17	INSERT, UPDATE, DELETE	Changing data safely
DAY 18	Constraints	PRIMARY KEY, FOREIGN KEY, UNIQUE, CHECK
DAY 19	Views	Saving a query as a virtual table
DAY 20	Indexes	Speeding up queries
DAY 21	Week 3 Review	Concept checkpoint

By the End of This Week, You'll Be Able To

- Design and create a new table with appropriate data types
- Safely insert, update, and delete rows without affecting unintended data
- Use constraints to prevent invalid data from ever entering a table
- Create a view to simplify a frequently-used complex query
- Explain, at a conceptual level, how an index speeds up a query

🎓 Teacher's Note

This week tends to be where students start to feel like real database owners rather than just query writers. Use that energy, but pair it with real caution: this is also the week where a careless UPDATE or DELETE can do genuine damage. I never let a student run an UPDATE or DELETE without first running the exact same WHERE clause as a SELECT — that one habit prevents nearly every serious mistake at this stage.

WEEK 3 — Building Databases • DAY 15 OF 30

Creating Tables: CREATE TABLE & Data Types

> **🎓 Teacher's Note**
> CREATE TABLE is where students first act as database designers rather than just query writers, and that shift deserves to be named explicitly. Choosing the right data type and constraints up front prevents entire categories of bad data later — a lesson usually learned the hard way in real jobs, so I try to teach it proactively here instead.

Designing a Table Before Creating It

Before writing CREATE TABLE, decide: what columns does this table need, and what **type** of data will each one hold? Getting this right up front prevents painful fixes later.

Common SQL Data Types

Type	Stores	Example
INTEGER	Whole numbers	25, -3, 1000
REAL / FLOAT / DECIMAL	Decimal numbers	19.99, 3.14
TEXT / VARCHAR(n)	Text of any length (or up to n characters)	'Aisha', 'Notebook'
DATE	A calendar date	'2026-08-25'
BOOLEAN	True/false (stored as 0/1 in SQLite)	TRUE, FALSE

The CREATE TABLE Statement

```SQL
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    age INTEGER,
    grade TEXT,
    enrolled_date DATE
);
```

Breaking Down the Syntax

Part	Meaning
id INTEGER PRIMARY KEY	A whole number column that uniquely identifies each row
name TEXT NOT NULL	A text column that can never be left empty
age INTEGER	A whole number column (NULL allowed unless stated otherwise)
enrolled_date DATE	Stores a calendar date

Auto-Incrementing Primary Keys

In SQLite, an INTEGER PRIMARY KEY column automatically increments on its own with every new row — you rarely need to specify an id manually.

```SQL
INSERT INTO students (name, age, grade) VALUES ('Aisha', 21, 'A');
-- id is assigned automatically: 1, then 2, then 3...
```

Verifying a Table Was Created

```SQL
SELECT * FROM students;
```

RESULT

id	name	age	grade	enrolled_date

0 rows — table exists but is empty

Creating a Table from Another Table's Structure

Useful for quickly making a backup or a temporary working copy.

```sql
CREATE TABLE students_backup AS
SELECT * FROM students;
```

⚠ **WATCH OUT:** Choosing TEXT for a column that should really be a number (like age or price) causes subtle bugs later — sorting and math on that column may behave unexpectedly. Match the data type to what the data actually represents, not just to what's easiest to type right now.

Planning Checklist Before Creating a Table

- ☐ What's the primary key, and will it auto-increment?
- ☐ Which columns should never be allowed to be empty (NOT NULL)?
- ☐ Which columns hold numbers vs text vs dates?
- ☐ Does this table need a foreign key connecting it to another table?

✏ **PRACTICE:** Design and create a 'books' table with columns for id (primary key), title (required text), author, price (a decimal number), and published_date. Insert 3 sample rows.

Quick Check

What's the difference between INTEGER and TEXT as a column type?
Answer: INTEGER stores whole numbers and allows mathematical operations and correct numeric sorting; TEXT stores characters/strings and sorts alphabetically, not numerically.

WEEK 3 — Building Databases • DAY 16 OF 30

Altering & Dropping Tables

🎓 Teacher's Note

ALTER TABLE feels minor, but it's genuinely one of the more dangerous commands in this course to run carelessly on a real production database — dropping a column is permanent and irreversible without a backup. I make a point of saying this out loud every single time I teach this lesson, because habits formed in a beginner's sandbox database quietly carry over to real ones later.

Why Tables Need to Change Over Time

Real databases evolve — a new business requirement means a new column, a mistake in the original design needs fixing, or an old, unused table needs to go. **ALTER TABLE** and **DROP TABLE** handle these changes.

Adding a Column

```sql
ALTER TABLE students
ADD COLUMN email TEXT;
```

```sql
SELECT * FROM students LIMIT 1;
```

RESULT

id	name	age	grade	enrolled_date	email
1	Aisha	21	A	2026-01-10	NULL

New column added, existing rows get NULL

Renaming a Column

```sql
ALTER TABLE students
RENAME COLUMN grade TO current_grade;
```

Renaming a Table

```sql
ALTER TABLE students
RENAME TO learners;
```

Dropping (Deleting) a Column

```sql
ALTER TABLE students
DROP COLUMN email;
```

⚠ **WATCH OUT:** Dropping a column permanently deletes all data stored in it — there's no undo. Always back up important data (or confirm you truly don't need it) before running DROP COLUMN on a real database.

Dropping an Entire Table

```sql
DROP TABLE students_backup;
```

⚠ **WATCH OUT:** DROP TABLE deletes the table AND every row inside it, permanently. This is one of the most dangerous commands in SQL — many teams restrict who is allowed to run it on production databases.

Using IF EXISTS for Safety

Adding **IF EXISTS** prevents an error if the table or column you're trying to remove doesn't actually exist — useful in scripts that might run more than once.

```sql
DROP TABLE IF EXISTS students_backup;
```

A Safer Pattern: Rename Before You Drop

In real teams, instead of immediately dropping something important, it's common to rename it first (e.g. old_students) and leave it for a week or two before permanently deleting it — a cheap safety net against a change that turns out to be a mistake.

ALTER TABLE Command Summary

Command	Purpose
ADD COLUMN	Adds a new column to an existing table
DROP COLUMN	Permanently removes a column and its data
RENAME COLUMN ... TO ...	Renames a column
RENAME TO	Renames the entire table
DROP TABLE	Permanently deletes a table and all its data

✏ **PRACTICE:** Using your 'books' table from yesterday, add a new column called 'genre', rename the 'author' column to 'author_name', then create a books_backup copy before practicing DROP TABLE IF EXISTS on it.

Quick Check

What's a safer alternative to immediately running DROP TABLE on something you're not 100% sure about?
Answer: Rename it first (e.g. to old_table_name) and leave it for a while before permanently dropping it — giving yourself a recovery window if it turns out you still needed it.

WEEK 3 — Building Databases • DAY 17 OF 30

INSERT, UPDATE & DELETE

🎓 Teacher's Note

INSERT, UPDATE, and DELETE are where SQL stops being purely about reading data and starts being about changing it — and that shift in stakes deserves real caution. I insist every student write and mentally verify a SELECT with the same WHERE clause BEFORE running the matching UPDATE or DELETE, every single time, no exceptions, until it becomes an unbreakable habit.

INSERT — Adding New Rows

SQL

```
INSERT INTO students (name, age, grade)
VALUES ('Karan', 23, 'B');
```

SQL

```
INSERT INTO students (name, age, grade) VALUES
    ('Priya', 20, 'A'),
    ('Dev', 22, 'C'),
    ('Sana', 21, 'B');
```

You can insert one row or many rows in a single statement — inserting several at once is more efficient than running INSERT repeatedly.

UPDATE — Modifying Existing Rows

SQL

```
UPDATE students
SET grade = 'A'
WHERE name = 'Karan';
```

SQL

```
UPDATE students SET grade = 'A' WHERE name = 'Karan';
```

OUTPUT

```
1 row updated
```

⚠ **WATCH OUT:** Running UPDATE without a WHERE clause changes EVERY row in the table — a devastating and common mistake. Always write and mentally verify your WHERE clause (ideally test it as a SELECT first) before running an UPDATE.

The Golden Rule: SELECT Before You UPDATE or DELETE

SQL

```
-- Step 1: Check exactly which rows will be affected
SELECT * FROM students WHERE grade = 'C';
```

SQL

```
-- Step 2: Only then run the UPDATE with the SAME WHERE clause
UPDATE students SET grade = 'B' WHERE grade = 'C';
```

Updating Multiple Columns at Once

SQL

```
UPDATE students
SET grade = 'A', age = 24
WHERE name = 'Dev';
```

DELETE — Removing Rows

```sql
DELETE FROM students
WHERE name = 'Sana';
```

```sql
DELETE FROM students WHERE name = 'Sana';
```

```
OUTPUT
1 row deleted
```

⚠ **WATCH OUT:** Just like UPDATE, running DELETE FROM students without a WHERE clause deletes EVERY row in the table — instantly and (without a backup) permanently. Always double, even triple-check your WHERE clause first.

Deleting Every Row vs Dropping the Table

Command	Effect on structure	Effect on data
DELETE FROM table;	Table still exists, empty	All rows removed
DROP TABLE table;	Table no longer exists at all	All rows removed (with the table itself)

INSERT ... SELECT — Copying Data Between Tables

```sql
INSERT INTO students_backup (name, age, grade)
SELECT name, age, grade FROM students
WHERE grade = 'A';
```

This copies only the matching rows from one table directly into another — no need to manually retype the data.

💡 **TIP:** Many professional teams require a SELECT with the exact same WHERE clause to be run and reviewed before any UPDATE or DELETE touches a production database. Build that habit now, while mistakes are cheap.

✏ **PRACTICE:** On your books table, write an UPDATE that raises the price of one specific book by checking the SELECT first, then a DELETE that removes one specific book by id — verifying with SELECT before and after each step.

Quick Check

What happens if you run DELETE FROM students; with no WHERE clause?
Answer: Every single row in the students table is deleted — the table itself still exists, but it becomes completely empty.

WEEK 3 — Building Databases • DAY 18 OF 30

Constraints In Depth

Teacher's Note

Constraints often get taught as a dry list of keywords, but each one exists to prevent a specific, realistic mistake — I always pair every constraint with the exact bad data it's designed to block, rather than presenting the syntax in isolation. A constraint that has no obvious enforcement value to a student is a constraint they'll be tempted to skip in their own projects.

What Are Constraints?

A **constraint** is a rule enforced directly by the database that prevents invalid data from ever being inserted or updated — a much stronger guarantee than hoping every application remembers to validate data correctly.

PRIMARY KEY — Uniquely Identifies Each Row

```sql
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT
);
```

A primary key is automatically unique and can never be NULL.

NOT NULL — This Column Can Never Be Empty

```sql
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL
);
```

```sql
INSERT INTO students (id, name) VALUES (1, NULL);
```

```
Error: NOT NULL constraint failed: students.name
```

UNIQUE — No Duplicate Values Allowed

```sql
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    email TEXT UNIQUE
);
```

```sql
INSERT INTO students (id, email) VALUES (1, 'a@x.com');
INSERT INTO students (id, email) VALUES (2, 'a@x.com');
```

```
Error: UNIQUE constraint failed: students.email
```

CHECK — Custom Validation Rules

```sql
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    age INTEGER CHECK (age >= 0 AND age <= 120)
);
```

```sql
INSERT INTO students (id, age) VALUES (1, -5);
```

```
Error: CHECK constraint failed: students
```

DEFAULT — A Fallback Value

```sql
CREATE TABLE students (
    id INTEGER PRIMARY KEY,
    name TEXT,
    status TEXT DEFAULT 'Active'
);
```

```sql
INSERT INTO students (id, name) VALUES (1, 'Aisha');
SELECT * FROM students;
```

RESULT

id	name	status
1	Aisha	Active

status filled in automatically, even though it wasn't provided

FOREIGN KEY — Enforcing Relationships

```sql
CREATE TABLE orders (
    id INTEGER PRIMARY KEY,
    student_id INTEGER,
    FOREIGN KEY (student_id) REFERENCES students(id)
);
```

```sql
INSERT INTO orders (id, student_id) VALUES (1, 999);
```

OUTPUT

```
Error: FOREIGN KEY constraint failed (no student with id 999 exists)
```

This prevents "orphaned" rows — an order pointing to a student that doesn't actually exist.

Constraint Quick Reference

Constraint	Prevents
PRIMARY KEY	Duplicate or missing unique identifiers
NOT NULL	Missing required data
UNIQUE	Duplicate values in a column that should be one-of-a-kind
CHECK	Values that don't satisfy a custom condition
DEFAULT	Not exactly a restriction — fills in a value automatically when none is given
FOREIGN KEY	References to rows that don't exist in another table

💡 **TIP:** Add constraints while DESIGNING a table, not after data quality problems have already occurred. It's far easier to prevent bad data from entering a database than to clean it up after the fact.

✏ **PRACTICE:** Redesign your books table with constraints: id as PRIMARY KEY, title as NOT NULL, an isbn column as UNIQUE, and a price column with a CHECK ensuring it's greater than 0.

Quick Check

What's the difference between UNIQUE and PRIMARY KEY?
Answer: A table can have only ONE primary key (and it can never be NULL), but it can have MULTIPLE unique columns, and unique columns (unless also marked NOT NULL) can allow NULL values.

WEEK 3 — Building Databases • DAY 19 OF 30

Views: Saving a Query as a Virtual Table

🎓 **Teacher's Note**

Views are conceptually simple — a saved query — but their real value only becomes obvious once students have already written the same complicated JOIN three times in three different queries. I try to make sure that repetition is genuinely felt as annoying before introducing views as the fix, rather than presenting them as a good idea in the abstract.

The Problem Views Solve

Imagine writing the same complicated 3-table JOIN every single day for a report. Retyping it repeatedly is tedious and error-prone — a **VIEW** saves a query under a name, so you can treat it like a simple table from then on.

Creating a View

```sql
CREATE VIEW active_loans AS
SELECT b.title, m.name AS borrowed_by, l.loan_date
FROM loans AS l
INNER JOIN books AS b ON l.book_id = b.id
INNER JOIN members AS m ON l.member_id = m.id
WHERE l.return_date IS NULL;
```

Querying a View — Just Like a Table

```sql
SELECT * FROM active_loans;
```

RESULT

title	borrowed_by	loan_date
SQL for All	Aisha	2026-02-01
Data Stories	Ravi	2026-02-10

From here on, anyone can run SELECT * FROM active_loans without needing to know or re-write the underlying 3-table JOIN.

Filtering and Sorting a View Further

```sql
SELECT * FROM active_loans
WHERE borrowed_by = 'Aisha';
```

RESULT

title	borrowed_by	loan_date
SQL for All	Aisha	2026-02-01

A view behaves like a real table for querying purposes — you can add WHERE, ORDER BY, or JOIN it with other tables.

Why Use a View?

Benefit	Explanation
Simplifies repeated queries	Write a complex JOIN once, reuse it everywhere
Improves readability	SELECT * FROM active_loans reads far more clearly than the full JOIN
Adds a layer of security	You can grant access to a view without exposing all columns of the underlying tables
Keeps logic consistent	Everyone uses the same definition of 'active loan,' instead of rewriting slightly different versions

A View Is NOT a Copy of the Data

A view doesn't store data itself — it re-runs its underlying query every time you access it. If the underlying tables change, the view's results automatically reflect that, with nothing to manually refresh.

Dropping a View

```sql
DROP VIEW IF EXISTS active_loans;
```

Views vs Tables

	Table	View
Stores actual data?	Yes	No — it's a saved query
Updates automatically when source data changes?	N/A (it IS the source)	Yes, always reflects current data
Can you INSERT into it directly?	Yes	Sometimes, with restrictions — varies by database

💡 **TIP:** A good rule of thumb: if you find yourself copy-pasting the same JOIN into three or more different queries, that's a strong signal it belongs in a view instead.

✏ **PRACTICE:** Using your Library Management tables from Project 1, create a view called overdue_summary that shows books currently on loan for more than 14 days, then query it with an additional WHERE filter.

Quick Check

Does a view store its own copy of data?
Answer: No — a view is a saved query that re-runs against the underlying tables every time it's accessed, so it always reflects the current data.

WEEK 3 — Building Databases • DAY 20 OF 30

Indexes & Query Performance Basics

🎓 **Teacher's Note**

Indexes are the first genuinely 'behind the scenes' topic in this course — invisible to a query's result, but critical to its speed. I keep this lesson deliberately conceptual rather than deeply technical: the goal is an accurate mental model (an index is like a book's index, not the book itself), not query-planner expertise, which takes years to develop fully.

Why Queries Can Be Slow

Without help, a database often has to check **every single row** in a table to answer a query — fine for a hundred rows, painfully slow for ten million. An **index** is a special data structure that lets the database find matching rows far faster, similar to how a book's index lets you jump straight to a topic instead of reading every page.

Creating an Index

```sql
CREATE INDEX idx_students_name
ON students(name);
```

This builds a fast lookup structure specifically for the **name** column, speeding up any query that filters or sorts by it.

Where Indexes Help Most

Query pattern	Why an index helps
WHERE email = 'a@x.com'	Jumps straight to matching rows instead of scanning every row
ORDER BY last_name	Data can already be in sorted order, avoiding a full sort
JOIN ... ON a.id = b.a_id	Speeds up matching rows between the two tables

Using EXPLAIN to See What's Happening

```sql
EXPLAIN QUERY PLAN
SELECT * FROM students WHERE name = 'Aisha';
```

```sql
EXPLAIN QUERY PLAN SELECT * FROM students WHERE name = 'Aisha';
```

```
SEARCH students USING INDEX idx_students_name (name=?)
```

This confirms the database is actually using your index, instead of scanning the whole table.

Indexes Aren't Free

Cost	Why it matters
Extra storage space	Each index is essentially a separate copy of that column's data, specially organized
Slower writes	Every INSERT/UPDATE/DELETE must also update every index on that table
Maintenance overhead	More indexes = more bookkeeping for the database on every change

When to Add an Index

Good candidate for an index	Poor candidate for an index
Columns frequently used in WHERE, JOIN, or ORDER BY	Columns rarely queried directly
Large tables (thousands+ of rows)	Very small tables (the whole table fits in memory instantly anyway)
Foreign key columns (often searched via JOIN)	Columns that change extremely often

Primary Keys Are Automatically Indexed

You never need to manually index a PRIMARY KEY column — the database creates that index automatically, since uniqueness checks require fast lookups anyway.

⚠ **WATCH OUT:** Adding an index to every column 'just in case' is a common beginner overcorrection — it slows down every future write without meaningfully speeding up queries that never actually use that column. Add indexes deliberately, based on real, frequently-run queries.

💡 **TIP:** Think of an index the way you'd think of a book's index: incredibly useful for the handful of terms you actually look up, and pointless (or even counterproductive) to build one for every single word in the book.

✏ **PRACTICE:** On your students table, create an index on the 'grade' column, then use EXPLAIN QUERY PLAN to confirm a query filtering WHERE grade = 'A' actually uses it.

Quick Check

What's the main trade-off of adding an index?
Answer: Faster reads (SELECT queries) in exchange for slower writes (INSERT/UPDATE/DELETE) and extra storage space, since the index has to be maintained alongside the table.

WEEK 3 — Building Databases • DAY 21 OF 30

Week 3 Review: Concept Checkpoint

🎓 **Teacher's Note**

By the end of Week 3, students can both query and modify a real database schema. What I watch for in the review isn't whether every syntax detail is memorized — it's whether a student can look at a new problem and correctly decide: does this need a new table, a new constraint, or just a different query? That judgment call is the real skill this week was building toward.

Week 3 Recap

Day	Skill
15	CREATE TABLE, choosing appropriate data types
16	ALTER TABLE, DROP TABLE, safe renaming
17	INSERT, UPDATE, DELETE — with the SELECT-first safety habit
18	PRIMARY KEY, FOREIGN KEY, NOT NULL, UNIQUE, CHECK, DEFAULT
19	Views — saving a complex query as a reusable virtual table
20	Indexes and the basics of query performance

Concept Checkpoint: Design and Build From Scratch

This short exercise combines every Week 3 topic — a great gauge of what's solid before Week 4's advanced material.

```sql
CREATE TABLE employees (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    department TEXT,
    salary REAL CHECK (salary > 0),
    email TEXT UNIQUE
);

INSERT INTO employees (name, department, salary, email) VALUES
    ('Aisha', 'Engineering', 85000, 'aisha@company.com'),
    ('Ravi', 'Sales', 62000, 'ravi@company.com'),
    ('Meera', 'Engineering', 91000, 'meera@company.com');

CREATE INDEX idx_employees_department ON employees(department);

CREATE VIEW engineering_team AS
SELECT name, salary FROM employees
WHERE department = 'Engineering';

UPDATE employees SET salary = 95000 WHERE name = 'Meera';

SELECT * FROM engineering_team ORDER BY salary DESC;
```

```sql
SELECT * FROM engineering_team ORDER BY salary DESC;
```

RESULT

name	salary
Meera	95000
Aisha	85000

Notice how many Week 3 ideas appear in this one short script: a **CREATE TABLE** with multiple constraints, an **INDEX** for a frequently filtered column, a **VIEW** simplifying a common query, and an **UPDATE** whose effect is immediately visible through the view.

> 💡 **TIP:** If any single piece of that script felt unfamiliar, that's exactly the day worth revisiting before moving into Week 4 — window functions, CTEs, and transactions all assume this foundation is solid.

> ✏ **PRACTICE:** Extend the employees table with a CHECK constraint ensuring department is one of a fixed set of values ('Engineering', 'Sales', 'Marketing'), then create a second view showing only employees earning above the company average salary.

WEEK 4

Advanced SQL & Database Design

Days 22–30: Master window functions, CTEs, transactions, and stored logic — then design and build a complete capstone database.

Why This Week Matters

The final week is less about new basic syntax and more about the tools that separate someone who can write queries from someone who can design and maintain a real production database. Window functions and CTEs make complex analytical queries readable. Transactions keep multi-step changes safe. Normalization theory explains WHY well-designed databases are split the way they are. This is also the least hand-held week — real database problems never arrive with a lesson number attached to them.

This Week's Lessons

DAY 22	Window Functions	ROW_NUMBER, RANK, PARTITION BY
DAY 23	CTEs	The WITH clause, recursive CTEs
DAY 24	CASE Statements	Conditional logic inside a query
DAY 25	Transactions	COMMIT, ROLLBACK, ACID
DAY 26	Stored Procedures & Functions	Reusable server-side logic
DAY 27	Triggers	Automatic actions on data changes
DAY 28	Database Design & Normalization	1NF through 3NF
DAY 29	Query Optimization	EXPLAIN and best practices
DAY 30	Project 2	Capstone: E-Commerce Database

By the End of This Week, You'll Be Able To

- Use window functions to rank or compare rows without collapsing them
- Write a CTE to break a complex query into clear, named, readable steps
- Wrap multi-step changes in a transaction so they either fully succeed or fully fail
- Explain what normalization is and why it prevents data anomalies
- Design and build a complete, multi-table database from a set of requirements

> **🎓 Teacher's Note**
>
> By this point, most of the remaining struggle isn't about understanding any single feature — it's about deciding which tool to reach for first when a real, messy problem shows up with no instructions attached. That's exactly what the Day 30 capstone project is designed to test, and it's the truest measure of whether the last four weeks actually stuck.

WEEK 4 — Advanced SQL • DAY 22 OF 30

Window Functions: ROW_NUMBER, RANK, PARTITION BY

> **🎓 Teacher's Note**
>
> Window functions are the topic most likely to make an intermediate student feel like a beginner again, right after they'd started feeling confident with joins and grouping. I always connect it back to Week 1's GROUP BY: a window function does something similar — calculating across a set of rows — but crucially without collapsing those rows into one, which is the entire point and the entire source of confusion.

What Problem Do Window Functions Solve?

GROUP BY collapses many rows into one summary row per group. A **window function** calculates something across a set of related rows too — but WITHOUT collapsing them. Every original row stays visible, with the calculated value attached alongside it.

Example Table: sales

sales

id	salesperson	region	amount
1	Aisha	North	5000
2	Ravi	North	7000
3	Meera	South	4000
4	Karan	South	6000

ROW_NUMBER() — Assigning a Sequential Number

```sql
SELECT salesperson, amount,
  ROW_NUMBER() OVER (ORDER BY amount DESC) AS rank_overall
FROM sales;
```

RESULT

salesperson	amount	rank_overall
Ravi	7000	1
Karan	6000	2
Aisha	5000	3
Meera	4000	4

PARTITION BY — Restarting the Calculation Per Group

PARTITION BY is like GROUP BY, but for window functions — it restarts the calculation separately within each group, while still keeping every row.

```sql
SELECT salesperson, region, amount,
  ROW_NUMBER() OVER (PARTITION BY region ORDER BY amount DESC) AS rank_in_region
FROM sales;
```

RESULT

salesperson	region	amount	rank_in_region
Ravi	North	7000	1
Aisha	North	5000	2
Karan	South	6000	1
Meera	South	4000	2

Notice: the ranking restarts at 1 for each region, but every original row from the sales table is still fully visible.

RANK() and DENSE_RANK() — Handling Ties

```sql
SELECT salesperson, amount,
  RANK() OVER (ORDER BY amount DESC) AS rnk,
  DENSE_RANK() OVER (ORDER BY amount DESC) AS dense_rnk
FROM sales;
```

RESULT

salesperson	amount	rnk	dense_rnk
Ravi	7000	1	1
Karan	6000	2	2
Aisha	5000	3	3
Meera	4000	4	4

Function	Behavior on ties (e.g. two rows tied for 2nd place)
ROW_NUMBER()	Assigns 1, 2, 3, 4... arbitrarily breaking ties
RANK()	Assigns 1, 2, 2, 4 — skips a number after a tie
DENSE_RANK()	Assigns 1, 2, 2, 3 — does NOT skip a number after a tie

Running Totals with SUM() OVER()

```sql
SELECT salesperson, amount,
  SUM(amount) OVER (ORDER BY id) AS running_total
FROM sales;
```

RESULT

salesperson	amount	running_total
Aisha	5000	5000
Ravi	7000	12000
Meera	4000	16000
Karan	6000	22000

💡 **TIP:** The word OVER is the signal a function is being used as a window function rather than a regular aggregate — SUM(amount) alone collapses rows, but SUM(amount) OVER (...) keeps every row while still calculating across them.

✏ **PRACTICE:** Using the sales table, write a query that shows each salesperson's rank within their own region using RANK(), and a separate query showing a running total of sales ordered by id across the whole table.

Quick Check

What's the key difference between GROUP BY and a window function with PARTITION BY?
Answer: GROUP BY collapses rows into one row per group; a window function with PARTITION BY calculates across each group but keeps every original row visible.

WEEK 4 — Advanced SQL • DAY 23 OF 30

CTEs: The WITH Clause & Recursive Queries

🎓 Teacher's Note

CTEs (the WITH clause) are best introduced as 'a way to name a subquery so you can read it top to bottom instead of inside out.' Readability is genuinely the whole pitch — I show the identical query written both ways and just ask which one a colleague would rather inherit six months from now. The answer is never in doubt.

What Is a CTE?

A **CTE** (Common Table Expression), written with **WITH**, lets you name a subquery and reference it as if it were a temporary table

— making complex queries dramatically easier to read, top to bottom.

Without a CTE: Nested and Hard to Read

```sql
SELECT region, total
FROM (
    SELECT region, SUM(amount) AS total
    FROM sales
    GROUP BY region
) AS region_totals
WHERE total > 8000;
```

The Same Query, Written with a CTE

```sql
WITH region_totals AS (
    SELECT region, SUM(amount) AS total
    FROM sales
    GROUP BY region
)
SELECT region, total
FROM region_totals
WHERE total > 8000;
```

RESULT

region	total
North	12000
South	10000

Both queries return an identical result — but the CTE version reads top to bottom like a plan: "first calculate region totals, then filter them," instead of forcing you to read from the inside out.

Using Multiple CTEs Together

```sql
WITH region_totals AS (
    SELECT region, SUM(amount) AS total
    FROM sales
    GROUP BY region
),
top_region AS (
    SELECT region FROM region_totals
    ORDER BY total DESC
    LIMIT 1
)
SELECT s.salesperson, s.amount
FROM sales AS s
INNER JOIN top_region AS t ON s.region = t.region;
```

RESULT

salesperson	amount
Aisha	5000
Ravi	7000

Each CTE can reference the ones defined before it, letting you break a complicated question into clear, named, sequential steps.

Recursive CTEs — Working with Hierarchies

A **recursive CTE** repeatedly references itself, useful for hierarchical data like an org chart or category tree.

```sql
WITH RECURSIVE org_chain AS (
    SELECT id, name, manager_id, 1 AS level
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.id, e.name, e.manager_id, oc.level + 1
    FROM employees AS e
    INNER JOIN org_chain AS oc ON e.manager_id = oc.id
)
SELECT * FROM org_chain;
```

RESULT

id	name	manager_id	level
1	Priya (CEO)	NULL	1
2	Ravi	1	2
3	Aisha	1	2
4	Karan	2	3

The recursive CTE starts with the top of the hierarchy (no manager), then repeatedly finds the next level down, tracking depth along the way.

💡 **TIP:** Reach for a CTE any time a query needs a subquery that itself contains another subquery — nesting more than one level deep is exactly when readability starts to suffer without them.

✏ **PRACTICE:** Rewrite one of your Week 2 subquery exercises using a CTE instead, and compare the readability of both versions side by side.

Quick Check

What keyword starts a CTE?
Answer: WITH

WEEK 4 — Advanced SQL • DAY 24 OF 30

CASE Statements & Conditional Logic

🎓 **Teacher's Note**

CASE statements are where SQL briefly starts to feel like a real programming language, with actual conditional branching inside a single query. I connect it explicitly to the if/elif/else pattern many students have seen in other languages, since the underlying logic — check conditions top to bottom, stop at the first match — is identical.

Adding Conditional Logic to a Query

A **CASE** statement lets you build if/elif/else-style logic directly inside a SELECT — evaluating conditions top to bottom and returning the first matching result, just like a program's conditional branching.

Basic CASE Syntax

```sql
SELECT name, amount,
  CASE
    WHEN amount >= 6000 THEN 'High'
    WHEN amount >= 4500 THEN 'Medium'
    ELSE 'Low'
  END AS sale_tier
FROM sales;
```

```sql
SELECT salesperson, amount,
  CASE
    WHEN amount >= 6000 THEN 'High'
    WHEN amount >= 4500 THEN 'Medium'
    ELSE 'Low'
  END AS sale_tier
FROM sales;
```

RESULT

salesperson	amount	sale_tier
Aisha	5000	Medium
Ravi	7000	High
Meera	4000	Low
Karan	6000	High

CASE Inside an Aggregate — Conditional Counting

A very common real-world pattern: counting how many rows meet different conditions, all in a single query.

```sql
SELECT
  COUNT(CASE WHEN amount >= 6000 THEN 1 END) AS high_sales,
  COUNT(CASE WHEN amount < 6000 THEN 1 END) AS lower_sales
FROM sales;
```

RESULT

high_sales	lower_sales
2	2

CASE for Pivoting Data

```sql
SELECT
  SUM(CASE WHEN region = 'North' THEN amount ELSE 0 END) AS north_total,
  SUM(CASE WHEN region = 'South' THEN amount ELSE 0 END) AS south_total
FROM sales;
```

RESULT

north_total	south_total
12000	10000

This "pivots" data from rows into columns — one row, several summary columns — a technique often used to feed data directly into a report or chart.

CASE in ORDER BY — Custom Sort Order

```sql
SELECT name, current_grade FROM learners
ORDER BY
  CASE current_grade
    WHEN 'A' THEN 1
    WHEN 'B' THEN 2
    WHEN 'C' THEN 3
    ELSE 4
  END;
```

RESULT

name	current_grade
Aisha	A
Priya	A
Karan	B
Dev	C

Without CASE, ORDER BY current_grade would sort alphabetically (A, B, C) — which happens to work here, but CASE lets you define ANY custom order, like sorting sizes S/M/L/XL correctly instead of alphabetically.

Simple CASE vs Searched CASE

Form	Syntax	When to use
Simple CASE	CASE column WHEN value THEN ... END	Comparing one column to specific exact values
Searched CASE	CASE WHEN condition THEN ... END	Any condition — ranges, multiple columns, comparisons

💡 **TIP:** CASE statements inside SUM() or COUNT() are one of the most useful, most commonly seen patterns in real business reporting SQL — worth practicing until it feels natural.

✏ **PRACTICE:** Using the sales table, write a query with a CASE statement that labels each sale as 'Above Average' or 'Below Average' compared to the overall average amount (hint: you'll need a subquery for the average).

Quick Check

What does ELSE do in a CASE statement if you leave it out entirely?
Answer: Any row that doesn't match any WHEN condition returns NULL instead of a specified fallback value.

WEEK 4 — Advanced SQL • DAY 25 OF 30

Transactions: COMMIT, ROLLBACK & ACID

🎓 Teacher's Note

Transactions introduce a topic students usually haven't had to think about at all so far: what happens if something fails HALFWAY through a multi-step change. I like to use a bank transfer as the motivating example every single time — deducting money from one account but failing before crediting the other is a vivid, immediately understandable disaster that COMMIT and ROLLBACK exist specifically to prevent.

What Is a Transaction?

A **transaction** groups multiple SQL statements into one all-or-nothing unit — either every statement succeeds, or none of them take effect at all. This matters enormously whenever a real-world action requires several database changes to happen together correctly.

The Classic Example: A Bank Transfer

Transferring money requires TWO changes: subtract from one account, add to another. If the program crashes between the two steps, the money would simply vanish without a transaction protecting the operation.

```sql
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;

COMMIT;
```

COMMIT makes both changes permanent together. If anything goes wrong before COMMIT, you can undo everything with **ROLLBACK** instead.

ROLLBACK — Undoing an In-Progress Transaction

```sql
BEGIN TRANSACTION;

UPDATE accounts SET balance = balance - 500 WHERE id = 1;
-- Something goes wrong here — insufficient funds detected!
ROLLBACK;
```

```sql
ROLLBACK;
```

```
Transaction rolled back — the balance update above never actually took effect
```

ACID — The Four Guarantees of a Good Transaction

Letter	Property	Meaning
A	Atomicity	All statements succeed together, or none of them do
C	Consistency	The database moves from one valid state to another, never a broken in-between state
I	Isolation	Simultaneous transactions don't interfere with each other's in-progress changes
D	Durability	Once committed, changes survive even a crash immediately afterward

A Practical Pattern: Check, Then Act, Inside a Transaction

```sql
BEGIN TRANSACTION;

SELECT balance FROM accounts WHERE id = 1;
-- Application checks: is balance >= 500? If yes, continue:

UPDATE accounts SET balance = balance - 500 WHERE id = 1;
UPDATE accounts SET balance = balance + 500 WHERE id = 2;

COMMIT;
```

When You Actually Need a Transaction

Situation	Needs a transaction?
A single UPDATE statement	No — it's already atomic on its own
Multiple related INSERTs that must all succeed together	Yes
Transferring data between two tables where a partial result would be wrong	Yes
A simple read-only SELECT report	No

⚠ **WATCH OUT:** Forgetting to COMMIT leaves a transaction open indefinitely in some database systems — which can lock resources and cause serious problems for other users of the database. Always make sure every BEGIN TRANSACTION has a matching COMMIT or ROLLBACK.

💡 **TIP:** Whenever a real-world action naturally requires 'more than one thing to happen together, or none of them,' that's the signal a transaction is needed — the bank transfer example is the pattern to remember it by.

✏ **PRACTICE:** Simulate a bank transfer with an accounts table (id, name, balance). Write a transaction that moves money between two accounts, then write a second version that deliberately checks for insufficient funds and uses ROLLBACK instead of COMMIT.

Quick Check

What does the 'A' in ACID stand for, and what does it guarantee?

Answer: Atomicity — it guarantees that every statement in a transaction succeeds together, or none of them take effect at all.

WEEK 4 — Advanced SQL • DAY 26 OF 30

Stored Procedures & Functions

> **🎓 Teacher's Note**
> Stored procedures and functions are usually the point where a student starts to feel like they're programming inside the database itself, not just querying it. I keep this lesson intentionally narrow and practical — enough to read and write a simple one — rather than a deep dive, since the syntax varies meaningfully between database systems and over-teaching it here creates false confidence.

What Is a Stored Procedure?

A **stored procedure** is a saved, named block of SQL logic stored directly inside the database itself — similar in spirit to a function in Python, but living and running on the database server.

> ⚠ **WATCH OUT:** Syntax for stored procedures and functions varies significantly between database systems (MySQL, PostgreSQL, SQL Server) — SQLite does not support them natively. The examples below use MySQL-style syntax as a representative example.

Creating a Simple Stored Procedure (MySQL syntax)

```sql
DELIMITER //

CREATE PROCEDURE GetStudentsByGrade(IN target_grade VARCHAR(1))
BEGIN
    SELECT name, age FROM students WHERE grade = target_grade;
END //

DELIMITER ;
```

Calling a Stored Procedure

```sql
CALL GetStudentsByGrade('A');
```

RESULT

name	age
Aisha	21
Priya	20

The logic lives in one place inside the database — any application can call GetStudentsByGrade('A') without needing to know or rewrite the underlying query.

Why Use Stored Procedures?

Benefit	Explanation
Reusability	Write complex logic once, call it from many applications
Performance	Precompiled by the database, often faster than sending raw SQL repeatedly
Security	Applications can be given permission to CALL a procedure without direct table access
Consistency	Every caller gets identical, centrally-maintained logic

A Stored Function — Returns a Single Value

```sql
DELIMITER //

CREATE FUNCTION CalculateDiscount(price DECIMAL(10,2), percent INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN price - (price * percent / 100);
END //

DELIMITER ;
```

```sql
SELECT CalculateDiscount(1000, 20) AS discounted_price;
```

RESULT

discounted_price
800.00

Stored Procedure vs Function

	Stored Procedure	Function
Returns a value directly usable in SELECT?	No	Yes
Can perform INSERT/UPDATE/DELETE?	Yes	Typically restricted or not allowed
Called with	CALL procedure_name(...)	Used inside a query, like SELECT function_name(...)

When to Reach for a Stored Procedure

- Complex, multi-step logic that many different applications need to run identically
- Operations that should be tightly controlled and audited at the database level
- Performance-critical logic that benefits from running directly on the database server

💡 **TIP:** As a beginner, you don't need to master writing complex stored procedures immediately — recognizing what they are and roughly when they're used is the more valuable first step, since exact syntax varies so much between database systems.

✏ **PRACTICE:** Look up your specific database system's syntax for creating a simple stored procedure or function (search '[your database] create procedure tutorial'), and try creating one that returns students above a given age.

Quick Check

What's the main practical difference between a stored procedure and a stored function?
Answer: A function returns a single value and can be used directly inside a SELECT statement; a procedure is called separately with CALL and can perform actions like INSERT/UPDATE/DELETE.

WEEK 4 — Advanced SQL • DAY 27 OF 30

Triggers

🎓 **Teacher's Note**

Triggers are powerful enough that I always pair the lesson with an explicit warning: automatic, invisible behavior can be just as much a maintenance headache as a convenience if it's overused or poorly documented. I ask students to imagine debugging unexpected data changes with NO idea a trigger exists — that mental exercise alone tends to produce much more thoughtful, sparing trigger design.

What Is a Trigger?

A **trigger** is a block of SQL that runs **automatically** whenever a specific event happens on a table — an INSERT, UPDATE, or DELETE — without anyone needing to call it manually.

A Practical Example: Logging Changes

```sql
CREATE TABLE salary_log (
    id INTEGER PRIMARY KEY,
    employee_id INTEGER,
    old_salary REAL,
    new_salary REAL,
    changed_at TEXT
);

CREATE TRIGGER log_salary_change
AFTER UPDATE ON employees
FOR EACH ROW
WHEN OLD.salary != NEW.salary
BEGIN
    INSERT INTO salary_log (employee_id, old_salary, new_salary, changed_at)
    VALUES (OLD.id, OLD.salary, NEW.salary, datetime('now'));
END;
```

Seeing the Trigger Fire Automatically

```sql
UPDATE employees SET salary = 95000 WHERE name = 'Meera';
```

```sql
SELECT * FROM salary_log;
```

RESULT

id	employee_id	old_salary	new_salary	changed_at
1	3	91000	95000	2026-08-25 10:15:00

A log entry was created automatically — no explicit INSERT was written by hand

Nobody had to remember to log this change — the trigger handled it invisibly, the instant the UPDATE happened.

OLD and NEW — Referring to Before and After Values

Keyword	Refers to	Available in
OLD	The row's values BEFORE the change	UPDATE and DELETE triggers
NEW	The row's values AFTER the change	INSERT and UPDATE triggers

Common Trigger Use Cases

Use case	Trigger type
Automatically logging every change to sensitive data	AFTER UPDATE
Preventing a deletion under certain conditions	BEFORE DELETE
Automatically updating a 'last_modified' timestamp	BEFORE UPDATE
Keeping a running total or summary table in sync	AFTER INSERT / AFTER DELETE

BEFORE vs AFTER Triggers

Timing	Use when...
BEFORE	You want to validate or modify data before it's actually written
AFTER	You want to react to a change that has already happened, like logging it

⚠ **WATCH OUT:** Triggers execute invisibly — there's no explicit call to see in your application code. Overusing triggers, or stacking too many of them on one table, makes a database's behavior genuinely hard to trace and debug. Document every trigger clearly, and use them sparingly and deliberately.

Dropping a Trigger

```SQL
DROP TRIGGER IF EXISTS log_salary_change;
```

💡 **TIP:** Before creating a trigger, ask: could this logic live safely in the application code instead? Triggers are the right tool specifically when the rule MUST hold no matter which application or user touches the data — not simply as a convenience.

✏ **PRACTICE:** Create a trigger that prevents a DELETE on the employees table if the employee's salary is above 100000 (hint: use a BEFORE DELETE trigger that raises an error under that condition).

Quick Check

What's the difference between OLD and NEW inside a trigger?
Answer: OLD refers to the row's values before the change happened; NEW refers to the row's values after the change — OLD is unavailable in INSERT triggers, and NEW is unavailable in DELETE triggers.

WEEK 4 — Advanced SQL • DAY 28 OF 30

Database Design & Normalization (1NF–3NF)

Teacher's Note

Normalization is taught here at the END of the course rather than the beginning on purpose — the rules make far more sense once students have already felt the real pain of update anomalies and duplicated data firsthand, from the exercises in earlier weeks. Explaining 3NF to someone who's never actually suffered from data duplication is explaining a solution to a problem they haven't experienced yet.

What Is Normalization?

Normalization is a set of rules for organizing tables to reduce data duplication and prevent inconsistencies — the formal reasoning behind the "split data into related tables" idea from Day 8, taught now that you've felt the pain of poorly organized data firsthand.

The Problem: An Unnormalized Table

`orders (unnormalized)`

order_id	customer_name	customer_email	product	price
1	Aisha	aisha@x.com	Notebook	60
2	Aisha	aisha@x.com	Pen	10
3	Ravi	ravi@x.com	Laptop	55000

Aisha's name and email are duplicated. If her email changes, every single row must be updated — miss one, and the data becomes inconsistent.

First Normal Form (1NF): Atomic Values, No Repeating Groups

`Before 1NF — multiple values crammed into one cell`

student	subjects
Aisha	Math, Science, English

`After 1NF — one value per cell`

student	subject
Aisha	Math
Aisha	Science
Aisha	English

1NF rule: every column must hold a single, indivisible value — never a comma-separated list crammed into one cell.

Second Normal Form (2NF): No Partial Dependency

2NF applies to tables with a **composite** (multi-column) primary key — every non-key column must depend on the **entire** key, not

just part of it.

`Before 2NF`

student_id	course_id	student_name	grade
1	101	Aisha	A
1	102	Aisha	B

student_name only depends on student_id, not on the full (student_id, course_id) combination — a 2NF violation. The fix: move student_name into its own students table.

Third Normal Form (3NF): No Transitive Dependency

`Before 3NF`

employee_id	department_id	department_name
1	10	Engineering
2	10	Engineering
3	20	Sales

department_name depends on department_id, not directly on employee_id — a transitive dependency. The fix: move department_name into its own departments table, referenced by department_id.

`employees (3NF)`

employee_id	department_id
1	10
2	10
3	20

`departments (3NF)`

department_id	department_name
10	Engineering
20	Sales

Normal Forms Summary

Form	Rule	Fixes
1NF	Every column holds one atomic value	Comma-separated lists crammed into a cell
2NF	Every column depends on the WHOLE primary key	Data that only relates to part of a composite key
3NF	No column depends on another NON-key column	Data that should live in its own related table

When to Deliberately Break the Rules: Denormalization

Sometimes, for reporting or performance reasons, teams intentionally **denormalize** — accepting some duplication in exchange for faster, simpler read queries. This is a deliberate trade-off, made only after understanding what normalization would otherwise look like.

> 💡 **TIP:** Normalization theory makes far more intuitive sense once you've personally felt the pain of an update anomaly, like the Aisha email example — if this lesson feels abstract, go back and try updating duplicated data by hand first.

> ✏ **PRACTICE:** Take an unnormalized 'invoice' table containing customer name, customer address, product name, product price, and quantity all in one table, and redesign it into properly normalized tables up to 3NF.

Quick Check

What does 1NF require of every column value?

Answer: That it be atomic (indivisible) — a single value, never a list of multiple values crammed into one cell.

WEEK 4 — Advanced SQL • DAY 29 OF 30

Query Optimization & Writing Clean SQL

🎓 Teacher's Note

Query optimization is the topic where I most actively discourage premature effort — I tell students plainly: write the clear, correct query first, and only optimize once you can prove (using EXPLAIN) that a specific query is actually slow. Optimizing queries that were never a real bottleneck is one of the most common ways beginners waste time and make their SQL harder to read for no measurable benefit.

The Golden Rule of Optimization

Write the clear, correct query first. Only optimize once you can **prove** — using EXPLAIN — that a specific query is actually slow. Optimizing queries that were never a real bottleneck wastes time and often makes SQL harder to read for no measurable benefit.

Reading an Execution Plan

```sql
EXPLAIN QUERY PLAN
SELECT * FROM orders WHERE customer_id = 5;
```

```sql
EXPLAIN QUERY PLAN SELECT * FROM orders WHERE customer_id = 5;
```

```
SCAN orders  -- no index used, checking every row
```

"SCAN" means the database is checking every row one by one — a strong candidate for adding an index if this query runs often on a large table.

```sql
CREATE INDEX idx_orders_customer ON orders(customer_id);
```

```sql
EXPLAIN QUERY PLAN SELECT * FROM orders WHERE customer_id = 5;
```

```
SEARCH orders USING INDEX idx_orders_customer (customer_id=?)
```

"SEARCH ... USING INDEX" confirms the query now jumps straight to matching rows instead of scanning the whole table.

Common Optimization Techniques

Technique	Why it helps
Select only needed columns, not SELECT *	Less data to read and transfer, especially with wide tables
Add indexes on frequently filtered/joined columns	Avoids full table scans
Filter as early as possible (WHERE before JOIN where possible)	Reduces the number of rows later steps have to process
Avoid functions on indexed columns in WHERE (e.g. WHERE YEAR(date)=2026)	Functions on a column often prevent the index from being used
Use LIMIT when you only need a preview	Avoids computing and returning unnecessary rows

A Query That Defeats Its Own Index

```sql
-- This WON'T use an index on order_date efficiently:
SELECT * FROM orders WHERE YEAR(order_date) = 2026;
```

```sql
-- This WILL use an index on order_date:
SELECT * FROM orders
WHERE order_date >= '2026-01-01' AND order_date < '2027-01-01';
```

Wrapping an indexed column in a function often forces the database to evaluate that function on every single row, defeating the purpose of the index entirely.

Writing Readable, Maintainable SQL

Practice	Example
Consistent capitalization	SELECT, FROM, WHERE in uppercase; table/column names in lowercase
Meaningful aliases	AS total_revenue, not AS x
One clause per line for complex queries	Easier to scan and debug than one giant single-line query
Comments for non-obvious logic	-- excludes cancelled orders from the total

Before and After: Readability

```sql
SELECT c.name,SUM(o.amount) as t FROM customers c INNER JOIN orders o ON c.id=o.customer_id WHERE
o.status!='cancelled' GROUP BY c.name HAVING SUM(o.amount)>1000 ORDER BY t DESC;
```

```sql
SELECT
    c.name,
    SUM(o.amount) AS total_spent
FROM customers AS c
INNER JOIN orders AS o ON c.id = o.customer_id
WHERE o.status != 'cancelled'
GROUP BY c.name
HAVING SUM(o.amount) > 1000
ORDER BY total_spent DESC;
```

Both queries run identically — but the second is dramatically easier for a colleague, or future you, to read and safely modify.

> 💡 **TIP:** A query that's easy to read is also a query that's easy to correctly optimize later — clean structure and good performance tend to go hand in hand far more often than beginners expect.

> ✏ **PRACTICE:** Take one of your Project 1 Library Management queries and check its EXPLAIN QUERY PLAN. If it shows a full table SCAN on a large hypothetical table, decide which column you'd index and why.

Quick Check

What's the recommended order of operations when a query feels slow?
Answer: Confirm it's actually slow and identify why using EXPLAIN, THEN optimize — never optimize a query you merely assume is slow without evidence.

FINAL PROJECT • BEGINNER TO PRO

Project 2 (Capstone): E-Commerce Database

Project Overview

Your capstone is a complete **E-Commerce Database** — customers, products, orders, and order items, designed from scratch and queried with realistic business questions. This project draws on skills from **every single week** of this course: table design, constraints, joins, aggregation, subqueries, and window functions.

Requirements

- ☐ Design a properly normalized schema (at least 4 related tables)
- ☐ Every table has an appropriate PRIMARY KEY and necessary FOREIGN KEYs
- ☐ Use constraints (NOT NULL, CHECK, UNIQUE) to enforce data quality

- ☐ Populate each table with realistic sample data
- ☐ Answer at least 5 realistic business questions using SQL
- ☐ Use at least one JOIN, one aggregate + GROUP BY, one subquery or CTE, and one window function

Step 1: The Schema

```
customers                products
+----+-------+          +----+---------+-------+
| id | name  |          | id | name    | price |
+----+-------+          +----+---------+-------+

orders                   order_items
+----+-------------+     +----+----------+------------+----------+
| id | customer_id |     | id | order_id | product_id | quantity |
| id | order_date  |     +----+----------+------------+----------+
+----+-------------+

One order can contain MANY order_items (many-to-many between orders and products)
```

Step 2: Create the Tables

```SQL
CREATE TABLE customers (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    email TEXT UNIQUE NOT NULL
);

CREATE TABLE products (
    id INTEGER PRIMARY KEY,
    name TEXT NOT NULL,
    price REAL CHECK (price > 0)
);

CREATE TABLE orders (
    id INTEGER PRIMARY KEY,
    customer_id INTEGER NOT NULL,
    order_date TEXT NOT NULL,
    FOREIGN KEY (customer_id) REFERENCES customers(id)
);

CREATE TABLE order_items (
    id INTEGER PRIMARY KEY,
    order_id INTEGER NOT NULL,
    product_id INTEGER NOT NULL,
    quantity INTEGER CHECK (quantity > 0),
    FOREIGN KEY (order_id) REFERENCES orders(id),
    FOREIGN KEY (product_id) REFERENCES products(id)
);
```

Step 3: Insert Sample Data

```SQL
INSERT INTO customers (name, email) VALUES
    ('Aisha', 'aisha@x.com'), ('Ravi', 'ravi@x.com'), ('Meera', 'meera@x.com');

INSERT INTO products (name, price) VALUES
    ('Notebook', 60), ('Pen', 10), ('Laptop', 55000), ('Mouse', 500);

INSERT INTO orders (customer_id, order_date) VALUES
    (1, '2026-03-01'), (1, '2026-03-15'), (2, '2026-03-10');

INSERT INTO order_items (order_id, product_id, quantity) VALUES
    (1, 1, 3), (1, 2, 5), (2, 3, 1), (3, 4, 2);
```

Business Question 1: What Is Each Customer's Total Spend?

```sql
SELECT c.name, SUM(p.price * oi.quantity) AS total_spent
FROM customers AS c
INNER JOIN orders AS o ON c.id = o.customer_id
INNER JOIN order_items AS oi ON o.id = oi.order_id
INNER JOIN products AS p ON oi.product_id = p.id
GROUP BY c.name
ORDER BY total_spent DESC;
```

RESULT

name	total_spent
Ravi	1000
Aisha	230

Business Question 2: Which Customers Have Never Ordered?

```sql
SELECT c.name FROM customers AS c
LEFT JOIN orders AS o ON c.id = o.customer_id
WHERE o.id IS NULL;
```

RESULT

name
Meera

Business Question 3: What's the Best-Selling Product by Quantity?

```sql
SELECT p.name, SUM(oi.quantity) AS total_sold
FROM products AS p
INNER JOIN order_items AS oi ON p.id = oi.product_id
GROUP BY p.name
ORDER BY total_sold DESC
LIMIT 1;
```

RESULT

name	total_sold
Pen	5

Business Question 4: Rank Customers by Spend (Window Function)

```sql
SELECT c.name, SUM(p.price * oi.quantity) AS total_spent,
  RANK() OVER (ORDER BY SUM(p.price * oi.quantity) DESC) AS spend_rank
FROM customers AS c
INNER JOIN orders AS o ON c.id = o.customer_id
INNER JOIN order_items AS oi ON o.id = oi.order_id
INNER JOIN products AS p ON oi.product_id = p.id
GROUP BY c.name;
```

RESULT

name	total_spent	spend_rank
Ravi	1000	1
Aisha	230	2

Business Question 5: Above-Average Orders (CTE)

```sql
WITH order_totals AS (
    SELECT o.id, SUM(p.price * oi.quantity) AS order_value
    FROM orders AS o
    INNER JOIN order_items AS oi ON o.id = oi.order_id
    INNER JOIN products AS p ON oi.product_id = p.id
    GROUP BY o.id
)
SELECT * FROM order_totals
WHERE order_value > (SELECT AVG(order_value) FROM order_totals);
```

RESULT

id	order_value
3	1000

Self-Assessment Rubric (for teaching use)

Criteria	Points
Schema is properly normalized with correct keys and constraints	25
All 5 business questions are answered correctly	30
At least one JOIN, one GROUP BY, one CTE/subquery, one window function used	25
SQL is clean, consistently formatted, and readable	10
Bonus: added a view, an index, or a trigger	10

Extend the Project (Recommended)

- ☐ Add a reviews table (many-to-one with products) and find each product's average rating
- ☐ Create a view called customer_order_summary combining Business Questions 1 and 2
- ☐ Add an index on order_items.order_id and confirm with EXPLAIN that it's used
- ☐ Add a trigger that prevents an order_item's quantity from being updated to 0 or less

💡 **TIP:** This capstone is intentionally open-ended past the core requirements. If you're teaching this course, treat the extensions as the real differentiator between students — anyone can follow the steps above, but choosing and correctly implementing a good extension shows real database design judgment.

30-Day Journey Recap

Week	Focus
Week 1	SELECT, WHERE, sorting, NULL, aggregate functions, GROUP BY
Week 2	Keys, all JOIN types, subqueries — and Project 1: Library Management
Week 3	CREATE/ALTER TABLE, INSERT/UPDATE/DELETE, constraints, views, indexes
Week 4	Window functions, CTEs, transactions, normalization — and Project 2: E-Commerce Database

Where to Go Next

- Practice on real public datasets using a free tool like DB Browser for SQLite or a cloud-hosted PostgreSQL database
- Learn a specific database system in more depth — PostgreSQL and MySQL each have valuable specialized features
- Explore how SQL connects to real applications via a language like Python (using libraries like sqlite3 or SQLAlchemy)
- Study query execution plans more deeply if performance tuning interests you
- Teach someone else — explaining a concept is the fastest way to master it

✏ **PRACTICE:** Build the complete E-Commerce Database exactly as shown, run all 5 business questions, then implement at least one extension from the list above on your own.